INVENTORY MANAGEMENT

MR.PIBHU CHITBURANACHART
MR.PANNAWISH KRIENGYAKUL
MR.AKKHRAWIN NAIR

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

FACULTY OF ENGINEERING & INTERNATIONAL COLLEGE
MAHIDOL UNIVERSITY

2026

INVENTORY MANAGEMENT

MR.PIBHU CHITBURANACHART
MR.PANNAWISH KRIENGYAKUL
MR.AKKHRAWIN NAIR

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENT FOR THE DEGREE BACHELOR OF ENGINEERING IN COMPUTER ENGINEERING

FACULTY OF ENGINEERING & INTERNATIONAL COLLEGE
MAHIDOL UNIVERSITY

2026

Computer Engineering Project

entitled

INVENTORY MANAGEMENT

Mr.Pibhu Chitburanachart
Researcher

Mr.Pannawish Kriengyakul
Researcher

Mr.Akkhrawin Nair
Researcher

Asst. Prof. Mingmanas Sivaraksa, Ph.D. (Information Engineering)
Advisor

Computer Engineering Project
entitled
INVENTORY MANAGEMENT

was submitted to the Faculty of Engineering & International College,
Mahidol University
for the degree of Bachelor of Engineering (Computer Engineering)
on
July 19, 2026

____

Asst. Prof. Mingmanas Sivaraksa, Ph.D. (Information Engineering)
Committee Chair

____

Asst. Prof. Lalita Narupiyakul, Ph.D. (Computer Science)
Committee

____

Asst. Prof. Dr. Tuangyot Supeekit, Ph.D. (Industrial Engineering and Systems)
Committee

BIOGRAPHY

NAME	Mr.Pibhu Chitburanachart
DATE OF BIRTH	29 August 2004
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	098 494 7984
E-MAIL	Pibhu22@gmail.com

NAME	Mr.Pannawish Kriengyakul
DATE OF BIRTH	17 April 2003
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	0989705604
E-MAIL	pannawish.kri@gmail.com

BIOGRAPHY

NAME	Mr.Akkhrawin Nair
DATE OF BIRTH	22 September 2002
BIRTHPLACE	Bangkok, Thailand
EDUCATION BACKGROUND	Mahidol International College
MOBILE PHONE	0987819644
E-MAIL	akkhrawin22@gmail.com

ACKNOWLEDGEMENT

First and foremost, we would like to express our sincere gratitude to our advisor and committee members, Asst. Prof. Mingmanas Sivaraksa, Asst. Prof. Lalita Narupiyakul, and Asst. Prof. Dr. Tuangyot Supeekit. Their insight, advice, and knowledge were invaluable in helping us complete this project within the available time frame.

Besides our advisors, we would like to acknowledge the use of AI tools in this project. AI was used as a supporting tool for help, guidance, brainstorming, language refinement, and researching related ideas during the development and writing process. The final project decisions, implementation, testing, and submitted content remained the responsibility of the project team.

Finally, we would like to thank Mahidol University International College and the Faculty of Engineering, Mahidol University, for providing the academic environment and resources that supported this project.

Mr.Pibhu Chitburanachart
Mr.Pannawish Kriengyakul
Mr.Akkhrawin Nair

INVENTORY MANAGEMENT

STUDENTS	Mr. Pibhu Chitburanachart ICCI
	Mr. Pannawish Kriengyakul ICCI
	Mr. Akkhrawin Nair ICCI
DEGREE	Bachelor of Engineering (Computer Engineering)
PROJECT ADVISOR	Asst. Prof. Mingmanas Sivaraksa, Ph.D. (Information Engineering)
DATE OF GRADUATION	July 19, 2026

ABSTRACT

This project presents an Inventory Management System for SME trading businesses that buy from suppliers and resell to customers. The system brings products, categories, suppliers, customers, purchases, sales, quotations, billing notes, payment batches, credit notes, inventory checking, gross-margin visibility, dashboard summaries, and a read-only AI assistant into one web application.

The application was developed with React and Vite on the frontend, Django REST Framework on the backend, and MySQL as the database. The backend controls important rules such as stock calculation, sale stock validation, and eligibility for billing notes, payment batches, and credit notes.

The completed system improves stock and margin visibility and connects purchasing, sales, and financial document tracking. The AI assistant is limited to supported read-only questions about stock, partners, receivables, payables, and document references. When an OpenAI API key is configured, the model explains verified backend facts and adds a concise conclusion and highlights; otherwise, the system returns a deterministic local summary.

78 Pages

CONTENTS

LIST OF TABLES

Table **Page**

LIST OF FIGURES

CHAPTER 1
INTRODUCTION

1.1 Background

Small and medium-sized trading businesses often operate as intermediaries between suppliers and customers [5]. In daily work, the business has to buy products from suppliers, receive stock, sell products to customers, prepare documents, and check whether the sales price still gives a reasonable margin. Because of this, inventory management is not only about counting how many products are left in the warehouse. It also involves quotations, purchase orders, sales records, delivery status, receivables, payables, and stock availability.

Many small businesses still depend on spreadsheets, paper records, manual document folders, or general-purpose accounting tools to manage these tasks [1, 8]. This can work at the beginning, but it becomes harder when the business has more products, more suppliers, different product units, delayed purchases, cancelled items, customer billing, and supplier payments. It also becomes difficult to answer simple questions quickly, such as which products are low in stock, which sales can be billed, which purchases are ready for payment, or whether there is enough received stock to fulfill a sale.

This project was created to bring those daily tasks into one web application. The Inventory Management System focuses on the main workflow of a trading business: managing master data, recording purchases and sales, checking stock, preparing quotations, tracking billing notes and payment batches, handling credit notes, viewing dashboard summaries, and asking limited read-only questions through the AI assistant.

1.2 Objective

1. To develop a system that records purchase and sales transactions, including the current status of each process, with an attachment file function that links all supporting documents directly to the corresponding transactions.

2. To develop an inventory management system that displays the overall product stock in the warehouse and provides tools for calculating essential inventory information.
3. To integrate a read-only AI chatbot using the OpenAI API that explains backend-prepared facts when configured and retains a deterministic local summary when the API is unavailable.

1.3 Scope

The scope of this project is based on the features that are already implemented in the Inventory Management System codebase. The system covers the following areas:

1. **Master data management:** The system manages categories, products, suppliers, and customers. Product records include SKUs, previous SKUs, product names, categories, unit conversions, reorder levels, active or disabled status, image or PDF attachments, and supplier-specific sourcing information.
2. **Purchasing workflow:** The system records purchase transactions, supplier information, purchase line items, expected delivery dates, received dates, item statuses, VAT totals, bill discounts, supplier tax invoice references, uploaded documents, and payable totals. The backend keeps purchase status and purchase item status in sync.
3. **Sales workflow:** The system records sales transactions, customer information, customer purchase-order references, line items, delivery statuses, uploaded documents, VAT totals, discounts, billing-note links, credit-note links, and source quotation links. The backend checks stock before a sale can move into a stock-deducting status.
4. **Inventory and stock control:** Available stock is calculated from received purchase quantities minus committed sale quantities. Sale allocations use received purchase layers and support first-in, first-out allocation. The inventory page and dashboard show stock position, reorder information, demand, purchase pipeline, stock value, and product history.
5. **Quotation workflow:** The system stores quotations with customer and supplier references, validity information, shipping date, payment term, VAT mode, to-

tal amounts, line items, and supplier options. Quotations can also be linked to purchases and sales created from them.

6. **Finance workflow:** The system supports billing notes for customer receivables, payment batches for supplier payables, and credit notes for cancelled or returned sales lines. Backend eligibility endpoints prevent users from selecting invalid transactions for these finance documents.
7. **Dashboard, margin, and reporting:** The backend provides dashboard summary and segment endpoints for finance, trends, product movement, cashflow, order coverage, and gross-margin visibility. Product, quotation, and sales data retain the purchase-cost and selling-price information needed to review reseller margin.
8. **Authentication and user access flow:** The backend includes JWT login, refresh, and authenticated user profile endpoints. The frontend supports login, token refresh, logout, and guest mode. The system also includes a user-access page for managing users, roles, and role permissions. Backend-wide authentication enforcement is controlled by the INVENTORY_REQUIRE_AUTH setting, and protected API actions use Django model permissions when authentication enforcement is enabled.
9. **AI-assisted operation:** The system includes a text-based inventory assistant endpoint for read-only questions covering stock and fulfillment, customer or supplier summaries, receivables and payables with exception checks, and reference or line-item lookup. With an OpenAI API key, the model explains verified backend facts and returns a conclusion and question-specific highlights; without a key or after an API failure, the same endpoint returns its local summary. The system also includes an AI report page that generates printable business reports for a selected supplier, customer, or product.
10. **Language and settings:** The application supports English and Thai interface text. The Settings page allows the user to choose the interface language without changing the underlying business records.

1.4 Expected Results

The expected result of this project is a working web-based inventory management system that can record purchases and sales, track the status of each process, and attach supporting documents to the correct transaction. The system should also show stock levels based on received purchases and committed sales, so users can understand where the stock comes from and where it is used.

The system is also expected to make daily checking easier through dashboard summaries and the limited AI-assisted text interface. Users should be able to find supported information such as product stock, transaction references, receivables, payables, partner summaries, and stock exceptions without manually searching every page. Overall, the completed system should reduce manual checking, improve stock visibility, and support better daily decisions for an SME trading business.

1.5 Timeline

Table 1.1 Project Timeline

Plan	Timeline								
	2025		2026						
	Nov	Dec	Jan	Feb	Mar	Apr	May	Jun	Jul
1: Project Proposal									
2: Requirement Gathering									
3: Products Page Development									
4: Sales and Purchases Page Development									
5: Categories Page Development									
6: Supplier and Customer Page Development									
7: AI Chat and AI Report Page Development									
8: Testing									
9: Quotation, Billing Notes, and Credit Notes Page Development									
10: Dashboard and Inventory Page Development									
11: Administration System Development									
12: Final Draft									

CHAPTER 2
LITERATURE REVIEW

This chapter presents a review of references related to the Inventory Management System. The reviewed works cover trading business workflow, spreadsheet-based business records, web-based inventory systems, inventory management concepts, and AI-assisted query support. These topics are important because the project is not only a stock-counting system. It also connects supplier purchasing, customer sales, quotations, receivables, payables, dashboard summaries, and a limited read-only AI assistant.

2.1 North American Industry Classification System: Wholesale Trade [5]

The North American Industry Classification System describes wholesale trade as an intermediate step in the distribution of merchandise. Wholesalers are organized to sell or arrange the purchase and sale of goods for resale, durable non-consumer goods, and raw or intermediate materials. This reference supports the business background of this project because the system is designed for a trading business that works between suppliers and customers.

In this project, the same idea appears in the main workflow. The business buys products from suppliers, stores received stock, and sells products to customers. Therefore, the system needs to manage both sides of the transaction. Supplier data, purchase records, customer data, sales records, and stock availability must be connected so that the user can understand the full flow of goods and money.

2.2 Qtier-Rapor: Managing Spreadsheet Systems and Improving Corporate Performance, Compliance and Governance [1]

This paper discusses how businesses often use spreadsheet systems to fill gaps in their core administrative systems. Spreadsheets are flexible and easy to start with, but the paper explains that larger spreadsheet systems can become harder to manage and control. This is relevant to small and medium-sized businesses because many businesses begin with simple spreadsheets before moving to a more structured system.

The problem is similar to the motivation of this project. A trading business may be able to record products, purchases, sales, and payments in spreadsheets at first. However, when the number of products, suppliers, customers, and transaction statuses increases, the records become harder to check. This supports the need for a web application with a database, backend validation, and clearer workflow separation.

2.3 Sole Traders Still Not Prepared for Digital Tax Returns in April [8]

This article reports research from Sage about sole traders and their preparation for digital tax reporting. It shows that many small business owners still use older methods such as spreadsheets, bank statements, and pen-and-paper records for financial administration. Although the article focuses on tax reporting, it is useful for this project because it shows that manual and semi-manual record keeping is still common in small business environments.

For an inventory trading business, the same issue can affect daily operations. If stock, purchasing, sales, billing, and payment information are kept in separate files or folders, it becomes difficult to answer simple questions quickly. Examples include checking which products are low in stock, which sales can be billed, and which purchases are ready for payment. This project addresses that issue by keeping related records in one web-based system.

2.4 A Full-Stack Web Solution for Online FMCG Management System Using MERN: Design, Implementation, and Evaluation [3]

This study presents the development of a full-stack web solution for managing fast-moving consumer goods. It discusses a web-based system with inventory management, order processing, stock updates, billing, and user roles. The reference is related to this project because both systems focus on using a web application to reduce manual business work and improve access to operational records.

The main difference is the technology stack and project scope. The reviewed system uses the MERN stack, while this project uses React for the frontend, Django REST Framework for the backend, and MySQL for the relational database. This project also focuses specifically on SME trading workflows, including purchases, sales, quotations, billing notes, payment batches, credit notes, stock validation, dashboard summaries, and limited AI-assisted operation.

2.5 Essentials of Inventory Management [4]

This book explains that inventory management is more than counting available stock. It includes demand, replenishment, physical stock handling, inventory value, and the business decisions connected to stock. These ideas support the design of the inventory and dashboard features in this project.

In the implemented system, stock is calculated from received purchase quantities and committed sale quantities. The dashboard and inventory page also show reorder-related information, pending demand, pending purchases, average cost, stock value, and product movement. These features follow the general inventory management idea that a business should understand both the physical quantity of stock and the financial meaning of that stock.

2.6 Inventory and Production Management in Supply Chains [7]

Silver, Pyke, and Thomas present inventory planning as a set of connected decisions involving demand, replenishment, lead time, service level, stockout risk, and cost. This perspective is relevant to a reseller because the useful stock position is not limited to the quantity currently on hand; it also depends on customer demand and confirmed incoming supply.

The project applies this relationship in its inventory workspace. Available stock, pending sales, pending purchases, lead-time demand, safety stock, and recommended restock are shown together so the user can review both current availability and replenishment pressure. The system does not claim to optimize the entire supply chain, but it uses these established concepts to present traceable operational indicators.

2.7 Inventory and Business Calculation Basis

This section connects the reviewed inventory-management concepts to the calculation requirements used in the project. Equations 2.13 to 2.18 are based on demand, replenishment planning, safety stock, and inventory value concepts [4, 7]. The remaining equations document project-specific transaction and finance definitions; their implementation and validation are described in Chapters 3 and 4 rather than treated as findings from the literature.

2.7.1 Unit Conversion and Line Amounts

Products can have different purchase or sale units. The backend normalizes transaction quantities into the product base unit.

$$\text{Base Quantity} = \text{Quantity} \times \text{Conversion Factor} \tag{2.1}$$

Purchase and sale line amounts are calculated from quantity, unit price or unit cost, and any percentage discount chain. The frontend clamps each discount between 0% and 100%.

$$\text{Discounted Amount} = \text{Quantity} \times \text{Unit Price} \times \prod_{i=1}^{n} \left(1 - \frac{\text{Discount Percent}_i}{100}\right) \tag{2.2}$$

For purchases, *Unit Price* means unit cost. For sales and quotations, it means sale price. Sales also apply the bill-level discount to the line amount when the sale total is calculated.

2.7.2 VAT Calculation

The purchase, sale, and quotation forms support VAT-included and VAT-not-included modes using a 7% VAT rate.

$$\text{VAT Rate} = 0.07 \tag{2.3}$$

For VAT-included transactions:

$$\text{Total Before VAT} = \frac{\text{Item Total}}{1 + 0.07} \tag{2.4}$$

$$\text{VAT Amount} = \text{Item Total} - \text{Total Before VAT} \tag{2.5}$$

For VAT-not-included transactions:

$$\text{VAT Amount} = \text{Item Total} \times 0.07 \tag{2.6}$$

$$\text{Grand Total} = \text{Item Total} + \text{VAT Amount} \tag{2.7}$$

2.7.3 Current Stock

The backend calculates product stock from transaction status instead of storing a manual stock number on the product.

$$\begin{aligned}\text{Current Stock} = \max(0, &\text{ Received Purchase Base Quantity} \\ &- \text{Committed Sale Base Quantity})\end{aligned} \tag{2.8}$$

Received purchase quantity comes only from purchase items with received status. Committed sale quantity comes only from sale items with packed, shipped, or delivered status. Pending sales are shown separately as demand, but they do not reduce current stock until they reach a stock-deducting status.

2.7.4 Stock Validation and Allocation

Before a sale is allowed to commit stock, the backend compares requested committed quantity with available stock.

$$\text{Stock Issue} = (\text{Requested Quantity} > \text{Available Quantity}) \tag{2.9}$$

When stock is committed, the backend creates sale item allocations from received purchase layers. If the user does not choose stock layers manually, the backend uses first-in, first-out allocation. The received layers are ordered by received date, purchase transaction date, purchase creation date, and item ID.

$$\text{Allocation Quantity} = \min(\text{Remaining Sale Quantity}, \text{Layer Available Quantity}) \tag{2.10}$$

Manual allocation is also supported, but the selected layers must match the product, must already be received, must have enough remaining quantity, and must add up exactly to the sale item base quantity.

2.7.5 Inventory Dashboard Calculations

The backend stock report calculates several useful inventory values for the dashboard and inventory page. The reorder-related formulas are aligned with the inventory management concept that reorder planning depends on demand during lead time and safety stock [4].

$$\text{Average Unit Cost} = \frac{\text{Received Purchase Value}}{\text{Received Purchase Units}} \tag{2.11}$$

$$\text{Average Daily Demand} = \frac{\text{Sales History Units}}{\text{Sales History Days}} \tag{2.12}$$

$$\text{Lead Time Demand} = \text{Average Daily Demand} \times \text{Average Lead Time} \tag{2.13}$$

$$\text{Safety Stock} = \text{Average Daily Demand} \times \text{Safety Stock Days} \tag{2.14}$$

$$\text{Calculated Reorder Level} = \lceil \text{Lead Time Demand} + \text{Safety Stock} \rceil \tag{2.15}$$

$$\text{Order-up-to Level} = \text{Reorder Level} + \lceil \text{Lead Time Demand} \rceil \tag{2.16}$$

$$\begin{aligned} \text{Recommended Restock} = \max(0,& \\ &\text{Order-up-to Level} + \text{Pending Sales} \\ &+ \text{Oversold Units} \\ &- \text{Available Stock} - \text{Pending Purchases}) \end{aligned} \tag{2.17}$$

$$\text{Stock Value} = \text{Available Stock} \times \text{Average Unit Cost} \tag{2.18}$$

The inventory page also uses supplier cost history to show supplier options. Supplier options are ranked by the most recent unit cost and order count, so the user can see a practical reorder source.

2.7.6 Purchase Payable and Payment Batch Calculations

The backend keeps the purchase grand total as the original document total, but it calculates a separate payable amount when purchase items are cancelled.

$$\text{Payable Total} = \text{Grand Total} \times \frac{\text{Non-cancelled Line Amounts}}{\text{All Line Amounts}} \tag{2.19}$$

This allows the system to keep the original purchase document readable while still showing the correct amount that should be paid to the supplier. Payment batch totals are calculated from their line amounts.

$$\text{Payment Batch Total} = \sum \text{Payment Batch Line Amount} \tag{2.20}$$

Unpaid payment batch lines are synchronized when the related purchase payable total changes. Paid lines are left unchanged because they represent a completed financial record.

2.7.7 Billing Note and Credit Note Calculations

Billing note totals come from selected eligible sales. Credit notes store line amounts from eligible cancelled or returned sale items. When a billing note has active credit notes, the net amount shown in the billing detail is reduced by the credit total.

$$\text{Billing Net Amount} = \text{Billing Note Total} - \text{Active Credit Note Total} \tag{2.21}$$

This makes the receivable amount clearer when a customer has a return or cancellation adjustment.

2.7.8 Cashflow and Net Position

The dashboard cashflow segment groups open receivables and open payables by due date. It also calculates the open net position.

$$\text{Open Net Position} = \text{Open Accounts Receivable} - \text{Open Accounts Payable} \tag{2.22}$$

Overdue accounts receivable are open billing notes with expected payment dates before today. Overdue accounts payable are open payment batches with planned payment dates before today.

2.7.9 Quotation Validity

Quotation validity is calculated from the quotation date and validity setting. If the validity mode is calendar days, the valid-until date is the quotation date plus the selected number of days.

$$\text{Valid Until} = \text{Quotation Date} + \text{Validity Days} \tag{2.23}$$

If the validity mode is business days, weekends are skipped during the date addition. The no-valid-date option stores no valid-until date.

2.8 Aisha: A Custom AI Library Chatbot Using the ChatGPT API [2]

This article describes the development of a custom chatbot using the ChatGPT API for a university library. The chatbot was designed to answer user questions using prepared system information and a controlled application context. The reference is useful for this project because it shows how a chatbot can be connected to a specific domain instead of being used as a general-purpose conversation tool.

In this Inventory Management System, the AI assistant follows a limited and read-only scope. It can support questions about stock and fulfillment, customer or supplier summaries, receivables and payables, and reference or line-item lookup. It does not create, approve, edit, cancel, or delete records. This limitation is important because inventory and finance records must remain controlled by the main application workflow and backend validation.

The current implementation also uses a structured response schema for model-backed chat output. OpenAI's structured-output guidance describes JSON Schema as a way to constrain response fields [6]. In this project, that mechanism separates the answer, conclusion, and highlights while the backend-prepared presentation remains the factual source.

2.9 Summary

The reviewed references support the main direction of this project. The wholesale trade reference explains why a trading business needs to manage the connection between suppliers, products, and customers. The spreadsheet and small-business references show why manual records, spreadsheets, and separated files can become difficult to manage as business operations grow.

The web-based inventory system reference supports the idea of building an integrated application for stock and transaction workflows. The inventory-management books support the calculation and reporting features, especially stock availability, replenishment, safety stock, reorder level, and inventory value. The chatbot and structured-output references support a controlled AI-assisted interface with a clearly defined domain and response contract.

Based on these works, this project focuses on developing a practical web-based inventory management system for SME trading businesses. The system combines React, Django REST Framework, and MySQL to manage business records in a structured way. It also adds dashboard summaries and a read-only AI assistant to help users find supported operational information more quickly.

CHAPTER 3
METHODOLOGY

This chapter explains how the Inventory Management web application was developed. The focus is on the development approach, architecture, workflow design, validation method, testing method, and tools used in the project. In this project, we built the system as a React web application connected to a Django REST Framework backend and a MySQL database.

3.1 Development Approach

We developed the system step by step, starting from the data that every other workflow depends on. The first part was master data, such as products, categories, suppliers, and customers. After that, we developed the transaction modules: purchases, sales, and quotations. The finance-related modules, including billing notes, payment batches, and credit notes, were added after the main transactions were already working. Finally, we connected the dashboard, inventory workspace, AI pages, administration pages, and language settings to the backend-supported workflow.

The project was separated into three main layers:

1. **Frontend layer:** React and Vite were used to build the web pages, forms, filters, tables, modals, and tab navigation.
2. **Backend layer:** Django REST Framework was used to expose APIs, validate data, calculate stock, control eligibility rules, prepare dashboard and assistant responses, and support AI report generation when an API key is configured.
3. **Database layer:** MySQL was used through the Django ORM to store relational business records.

This structure helped keep the user interface separate from the business rules. The frontend guides the user, but the backend is still responsible for the important checks such as stock validation, transaction eligibility, and payable amount synchronization.

3.2 Requirement Areas and Web-App Pages

The web application is organized around the main work areas that a trading business needs each day. Each sidebar page has a specific purpose, and the pages are connected through shared product, supplier, customer, purchase, sale, and finance data.

Table 3.1 Requirement Areas and Implemented Pages

Requirement Area	**Web-App Page**	**Purpose in the System**
Operation summary	Dashboard	Shows stock, finance, cashflow, product movement, and order coverage summaries.
Inventory control	Inventory	Shows stock health, reorder information, supplier options, product details, and quick purchase-order preparation.
Assistant support	AI Chat	Answers supported read-only questions from verified backend facts; configured AI responses add a conclusion and question-specific highlights.
Report generation	AI Report	Generates printable supplier, customer, or product reports with metrics, charts, record tables, and AI-written analysis.
Purchasing	Purchases	Records purchase orders, line items, receiving status, documents, VAT totals, discounts, and payable values.
Supplier payment	Payment Batches	Groups eligible purchases for supplier payment and tracks paid or unpaid lines.
Quotation	Quotation	Prepares quotations with line items, supplier options, validity dates, and conversion to purchase or sale records.
Sales	Sales	Records customer sales, stock allocation, delivery status, documents, VAT totals, and credit-note links.

Table 3.2 Record and Support Page Scope

Requirement Area	Web-App Page	Purpose in the System
Customer billing	Billing Notes	Groups eligible sales into customer billing notes and tracks received or outstanding lines.
Returns and cancellation adjustment	Credit Notes	Creates credit notes from eligible cancelled or returned sale items.
Product records	Products	Manages product details, SKUs, image/PDF attachments, units, reorder levels, active status, suppliers, margin information, and history.
Category records	Categories	Manages product categories using a parent-child category tree.
Supplier records	Supplier	Manages supplier profile, contact information, procurement contact, branches, shipping addresses, and terms.
Customer records	Customer	Manages customer profile, contact information, branches, shipping addresses, and billing terms.
Administration and audit	User Access / Activity Log	Manages users, roles, and permissions, and provides authorized review of system activity.
Language preference	Settings	Switches the application interface between English and Thai.

3.3 System Architecture

The system uses a three-tier web architecture. React is responsible for the user interface, Django REST Framework provides the API and business rules, and MySQL stores the relational records. The frontend does not connect directly to the database. Every important action goes through the API.

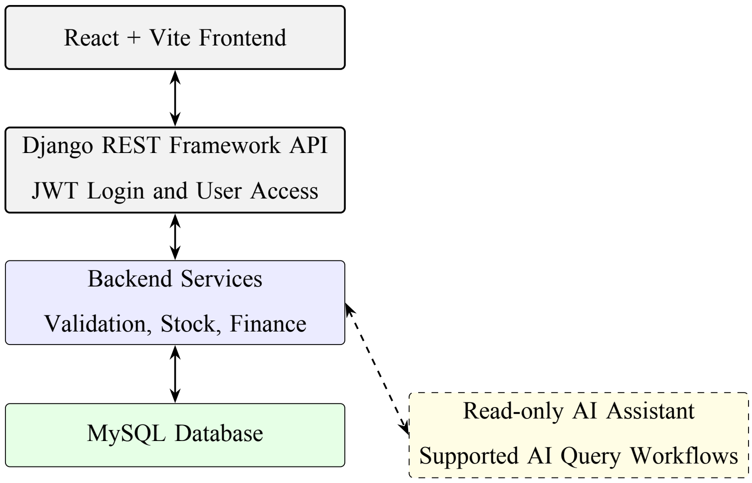

Figure 3.1 System Architecture Overview

Table 3.3 Architecture Components

Layer	**Implementation**	**Main Responsibility**
Frontend	React 18 and Vite	Builds the tabbed interface, forms, filters, tables, modals, and bilingual text display.
API	Django REST Framework	Receives requests, serializes data, applies filters, supports pagination, and returns JSON responses.
Business logic	Django service functions	Handles stock calculation, FIFO allocation, dashboard summaries, payable synchronization, eligibility checks, and assistant context.
Database	MySQL with Django ORM	Stores master data, transaction records, line items, documents, allocations, and finance records.
Authentication and access control	Simple JWT, Django groups, and Django permissions	Provides login, refresh token, authenticated profile support, role-based user access, and permission-gated API actions.

3.4 Frontend Methodology

The frontend was developed with React components, hooks, and helper files. Large pages are split into smaller sections, such as form details, line items, totals, directory tables, and detail modals. This made the code easier to manage because a purchase form, for example, has different concerns from a purchase history table.

The API client in frontend/src/api.js centralizes communication with the backend. It builds query strings, sends JSON or form-data requests, attaches bearer tokens, retries a request after token refresh, and dispatches an authentication-expired event when refresh fails. The frontend permission helper uses the profile returned from /api/auth/me/ to show or hide navigation tabs such as products, purchases, sales, user access, and activity log. The data-loading hooks keep full lookup data separate from paginated directory rows, so forms can still access reference data while history pages can use pagination. User-facing text is selected from English and Thai dictionaries through the language context, and the Settings page controls the active interface language.

The frontend also keeps a mock-data fallback for development and demonstration. This fallback is useful when the backend is not running, but it is not treated as the final authority. The backend remains the source of truth for stock validation, eligibility, and financial synchronization.

3.5 Backend and API Methodology

The backend was developed as a Django project with one main inventory application. Django REST Framework viewsets provide CRUD endpoints for products, categories, suppliers, customers, purchases, sales, quotations, billing notes, payment batches, and credit notes. Separate API views provide dashboard summaries, dashboard segments, lookup lists, product history, product stock layers, eligibility lists, and chat responses.

Table 3.4 Main API Endpoint Groups

Endpoint Group	Purpose
/api/products/, /api/categories/	Product catalog and category tree management.
/api/suppliers/, /api/customers/	Business partner master data management.
/api/purchases/, /api/sales/	Purchase and sales transaction workflows, including line items and documents.
/api/quotations/	Quotation records with normalized line items and supplier options.
/api/billing-notes/, /api/payment-batches/, /api/credit-notes/	Customer receivable, supplier payable, and credit note workflows.
/api/eligibility/...	Backend-controlled lists for billing-note sales, payment-batch purchases, and credit-note sale lines.
/api/dashboard/, /api/dashboard/segment/	Dashboard summary and segmented operational reports.
/api/products/<id>/history/ and /stock-layers/	Product transaction history and received stock layer details.
/api/chat/	Read-only assistant answers from backend-prepared facts. Configured OpenAI responses include a conclusion and highlights; failures use the local summary.
/api/ai-reports/	Generates printable HTML reports for a selected supplier, customer, or product with metrics, charts, and record tables.
/api/auth/login/, /refresh/, /me/	Login, token refresh, and authenticated user profile.
/api/admin/users/, /admin/roles/	User account management, role assignment, and role permission management for authorized administrators.
/api/activity-logs/	Read-only activity history for login, create, update, and delete events, available only to users with activity-log access.

Pagination is opt-in. If a page parameter is not sent, list endpoints can still return a plain array for compatibility with existing frontend logic. When pagination is requested, the response includes count, next, previous, page, page_size, total_pages, and results.

3.6 Database Design Methodology

The database was designed as a relational MySQL schema managed by Django migrations. Master data is separated from transaction data. For example, products, suppliers, customers, and categories are stored once and then referenced by purchases, sales, quotations, billing notes, payment batches, and credit notes.

The system also keeps snapshot fields inside transaction records. A sale item stores product name, SKU, quantity, price, and amount. A purchase stores supplier name and line item details. These snapshot values are important because old business documents should remain readable even if a product, supplier, or customer is edited later.

Table 3.5 Primary Database Model Groups

Group	**Models**
Master data	Category, Supplier, Customer, Product, ProductPicture, ProductUnitConversion, ProductSupplier
Purchasing	Purchase, PurchaseItem, PurchaseDocument
Sales	Sale, SaleItem, SaleItemAllocation, SaleDocument
Quotation	Quotation, QuotationItem, QuotationItemSupplier
Finance	BillingNote, BillingNoteLine, PaymentBatch, PaymentBatchLine, CreditNote, CreditNoteLine

3.7 Business Workflow Design

The main workflow starts from master data. Products belong to categories and are used in purchase, sale, and quotation line items. Purchases bring stock into the system only when purchase items are received. Sales commit stock only when sale items are packed, shipped, or delivered. This is why the system does not treat product stock as a manually edited field.

Quotations are used before purchase or sales records are created. Billing notes are created from eligible sales for customer receivable tracking. Payment batches are created from eligible purchases for supplier payable tracking. Credit notes are created from eligible cancelled or returned sale lines. Dashboard summaries, AI Chat, and AI Report read from the operational records, while user access and activity logs support administration and audit review.

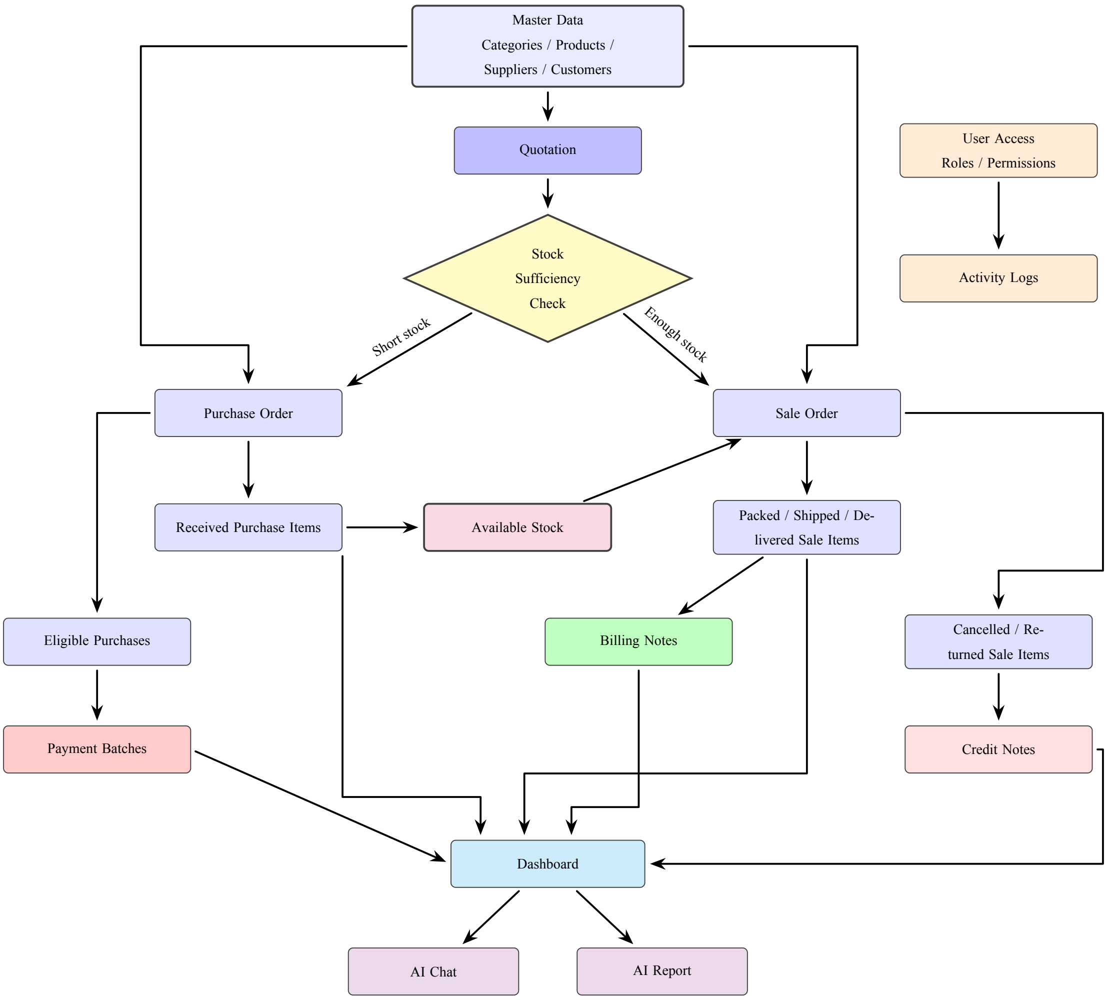

Figure 3.2 Business Workflow Design

3.8 Validation and Security Design

Validation is handled mainly by the backend. The frontend helps the user by showing form messages and previews, but direct API calls still need to be safe. The backend validates stock availability, active product usage, allocation rules, document eligibility, quotation validity dates, and finance document selection rules.

Security support is implemented with Simple JWT, Django groups, and Django model permissions. The backend exposes login, token refresh, and authenticated profile endpoints. The frontend stores and refreshes tokens, then attaches the access token to API requests. The authenticated profile includes the user's roles, permissions, and access flags for user management and activity logs. In local development, the backend can allow unauthenticated access depending on INVENTORY_REQUIRE_AUTH; when that setting is enabled, API access requires authentication and model-level permissions are checked for protected CRUD actions.

3.9 Testing Methodology

Testing was designed to check both technical correctness and user workflow understanding. The technical tests focused on backend behavior because the backend owns the most important business rules, including stock calculation, stock validation, finance eligibility, payable synchronization, and AI data preparation. Manual testing and questionnaires were added to check whether users could complete the main business workflow and understand the interface.

Table 3.6 Testing Summary

Test Area	**What It Checks**
Pagination and filtering	Paginated responses, page size limits, search, status filters, date filters, and product or partner filters.
Product data	Product image/PDF upload, active/disabled behavior, lookup data, stock filter, average cost, margin visibility, and product history.
Stock validation	Sales cannot commit more stock than available received purchase layers.
Stock allocation	FIFO allocation, manual allocation, and stock layer remaining quantity are handled correctly.
Reorder calculation	Fixed mock purchase and sale data is used to confirm available stock, average cost, lead time, daily demand, safety stock, reorder level, recommended restock, days until stockout, and stock value.
Finance eligibility	Billing notes, payment batches, and credit notes only use eligible records.
Financial synchronization	Purchase payable totals and payment batch line amounts stay consistent when purchases change.
User access control	Backend tests confirm that regular users cannot open administrator user endpoints, authorized administrators can create users and assign roles, and model permissions control protected API access when authentication enforcement is enabled.
Assistant scope	Supported questions return data-based answers, while unsupported questions are rejected as out of scope.
AI report output	Supplier, customer, and product report contexts are checked against backend data, and generated report HTML is sanitized and rendered with deterministic chart rows.

3.9.1 Automated Backend Validation Method

Automated backend validation was conducted with Django and Django REST Framework test cases. The tests used a temporary test database, created controlled records, called backend APIs or service functions, and compared the returned values with expected behavior. This method was used because backend rules must still be correct even if a user bypasses the frontend and calls the API directly.

The preserved black-box test history contains a selected 30-case workflow suite for presentation. That historical selection covered product validation, quotation behavior, purchase workflow, sales stock validation, finance eligibility, credit notes, payment batches, payable synchronization, and lookup behavior. It did not contain dedicated pagination or filtering cases. The final full backend suite contains 106 tests, including separate coverage for pagination, filtering, reorder calculation, AI Chat, AI Report, user access, permission enforcement, and activity logging. The test command used for the full backend suite was:

```
backend/.venv/bin/python backend/manage.py test inventory
```

Table 3.7 Automated Backend Test Method

Method Item	Description
Test data	Controlled products, suppliers, customers, purchases, sales, quotations, billing notes, payment batches, and credit notes were created inside the test database.
Test action	The test cases called backend endpoints or backend service functions, similar to how the frontend sends requests during real use.
Expected output	The returned API data, status codes, calculated values, and validation errors were compared with the expected business rules.
Access-control checks	User-access tests checked that ordinary users cannot open administrator endpoints, administrators can create users and assign roles, permissions control protected API access, and activity-log access is restricted.
Reason for method	Automated tests are repeatable and suitable for checking exact rules such as stock validation, eligibility, and financial synchronization.

3.9.2 Calculation Validation Method

The calculation validation focused on the reorder point and inventory dashboard calculation. The method used fixed mock data instead of live random data so that the expected output could be manually calculated and compared with the backend result.

The test created one product, received purchase stock, committed sales, pending sales, pending purchases, and lead-time samples. After the records were created, the dashboard stock report calculation was executed. The output was checked against expected values such as available stock, average unit cost, average daily demand, safety stock, reorder level, recommended restock, days until stockout, and stock value.

Table 3.8 Calculation Validation Method

Method Item	**Description**
Input data	Fixed purchase and sale records were created for a known product so the expected stock calculation could be reproduced manually.
Test action	The backend dashboard stock report logic was called using the controlled test records.
Expected output	The backend should return the same reorder, restock, stock value, and stockout values as the manual calculation.
Reason for method	Reorder point output affects purchasing decisions, so the formula must be validated using traceable input data.

3.9.3 AI Chat Validation Method

AI Chat validation was designed to confirm that the assistant answered from backend data and stayed within the supported assistant scope. The tests used known database records and sent fixed chat questions to the backend chat function. The checked output included the answer text, presentation title, referenced records, model marker, and the structured AI conclusion and highlights returned when a model was configured.

The method also tested unsupported questions. This was important because the assistant should not invent broad business reports or perform actions outside the current read-only assistant scope. When an API key setting was configured, the validation still checked that operational answers remained tied to backend summaries and selected records.

Table 3.9 AI Chat Validation Method

Method Item	**Description**
Input data	Known customer, sale, stock, billing, and payment records were prepared or selected from the backend context.
Test action	Fixed user questions were sent to AI Chat, such as customer activity, net position, low stock, and unsupported broad requests.
Expected output	Supported questions should return backend-based facts. A configured model should explain those facts and add a conclusion and highlights; local fallback should remain available, and unsupported questions should be rejected as outside scope.
Reason for method	AI Chat must help users read system data without inventing values or changing records.

3.9.4 AI Report Validation Method

AI Report validation checked whether report generation used the selected supplier, customer, or product records correctly. The method focused on the report context before and after the AI-written body was added. The backend prepared metrics, charts, record tables, and selected transaction references, then returned a self-contained printable HTML report.

The validation also checked report safety. Model-style HTML was simulated during testing so the system could verify that unsafe content was removed and that chart rows came from backend-prepared data. This makes the test repeatable and prevents the result from depending on changing AI output.

Table 3.10 AI Report Validation Method

Method Item	**Description**
Input data	A selected supplier, customer, or product was used with related purchases, sales, quotations, finance records, and stock data.
Test action	The report endpoint generated the report context and final HTML document for the selected scope.
Expected output	The report should contain correct selected-record references, summary metrics, backend chart rows, tables, a print button, and sanitized HTML.
Reason for method	AI Report may use AI for presentation, but the business values and charts must still come from reliable backend data.

3.9.5 Manual Scenario Test Method

Manual scenario testing was conducted to check whether users could complete the main business workflow in the system. The scenario was divided into parts that followed the intended workflow: quotation, purchase creation, purchase receiving, supplier payment, sale checking, status update, credit note, and billing note.

During the test, each participant followed the scenario steps and recorded the time and score for each part. Tester names were removed from the stored result file for privacy. This method was used because automated tests can prove that rules work, but they cannot show whether users can understand the screen flow and find the required actions.

Table 3.11 Manual Scenario Test Method

Method Item	**Description**
Input data	Testers used the prepared system data and followed scenario tasks covering the main transaction and finance workflow.
Test action	Testers completed each scenario part while completion time, task score, and comments were collected.
Expected output	Users should be able to complete the main workflow and identify where the system is easy or difficult to use.
Reason for method	Manual testing checks practical usability and workflow clarity from the user's point of view.

3.9.6 Questionnaire Method

After the scenario test, participants answered a questionnaire using a rating scale. The questionnaire measured their perception of usability, workflow clarity, system usefulness, and areas that may need improvement. Written comments were also reviewed to find repeated feedback themes.

The questionnaire was used as a supporting evaluation method. It did not replace automated validation, but it helped show whether the implemented system was understandable and useful to users after they tried the workflow.

Table 3.12 Questionnaire Method

Method Item	Description
Input data	Participants answered rating-scale questions and optional written feedback after completing the scenario test.
Test action	Scores were summarized, and comments were grouped into repeated themes such as easy areas, difficult areas, and visual clarity.
Expected output	The questionnaire should show general user perception and highlight improvement areas for the interface and workflow.
Reason for method	Questionnaire feedback adds user-centered evidence that cannot be captured by automated backend tests alone.

3.10 Tools and Technologies Used

The project used separate tools for frontend development, backend development, database storage, authentication, AI integration, styling, and testing. These tools were selected to support a full-stack web application with server-side business rules and a React-based user interface.

Table 3.13 Tools and Technologies

Category	**Tool or Technology**	**Use in This Project**
Frontend framework	React 18	Builds the web application screens and reusable UI components.
Frontend build tool	Vite	Runs the local development server and production build process.
Backend framework	Django 5	Provides project structure, ORM, migrations, settings, and backend application logic.
API framework	Django REST Framework	Provides serializers, viewsets, API views, JSON responses, parsers, and API tests.
Database	MySQL	Stores relational inventory, transaction, and finance data.
Authentication and access control	Simple JWT and Django permissions	Provides access and refresh token authentication, authenticated profiles, user-role management, permission checks, and activity-log access control.
AI integration	OpenAI API with local fallback	Uses structured model output for read-only chat explanations, conclusions, and highlights, and for AI report analysis when configured; backend-prepared local output remains available after missing keys or API failures.
Styling	CSS files by domain	Provides the compact inventory system interface.
Testing	Django TestCase and DRF APITestCase	Validates backend business rules and API behavior.

CHAPTER 4
RESULTS

This chapter explains the result of the implemented Inventory Management System. The result is based on the current codebase and implemented web-application pages. The calculation basis is defined in Chapter 2 and the development method in Chapter 3, so this chapter focuses on demonstrated workflows, evidence, and evaluation results.

4.1 System Overview

The final system is a full-stack web application for an SME trading business. It supports the flow from product setup to purchasing, selling, billing, supplier payment, credit note adjustment, inventory checking, dashboard reporting, and limited AI-assisted questions.

The main result is that the system connects business documents together instead of keeping them as separate records. A product can appear in purchases, sales, quotations, inventory reports, and product history. A sale can later become part of a billing note or a credit note. A purchase can later become part of a payment batch. This connection makes the system more useful than a simple product list.

4.2 Implemented Modules

All requirement areas mapped in Tables 3.1 and 3.2 were implemented. The completed workspace includes Dashboard, Inventory, AI Chat, AI Report, Purchases, Payment Batches, Quotation, Sales, Billing Notes, Credit Notes, Products, Categories, Supplier, Customer, User Access, Activity Log, and Settings. The following section presents page-level evidence without repeating the full requirement map.

4.3 System Implementation

This section combines the page-development explanation with selected screenshots from the finished application. Each screenshot is paired with a description of what the page does, how it was developed, and why it matters for day-to-day use.

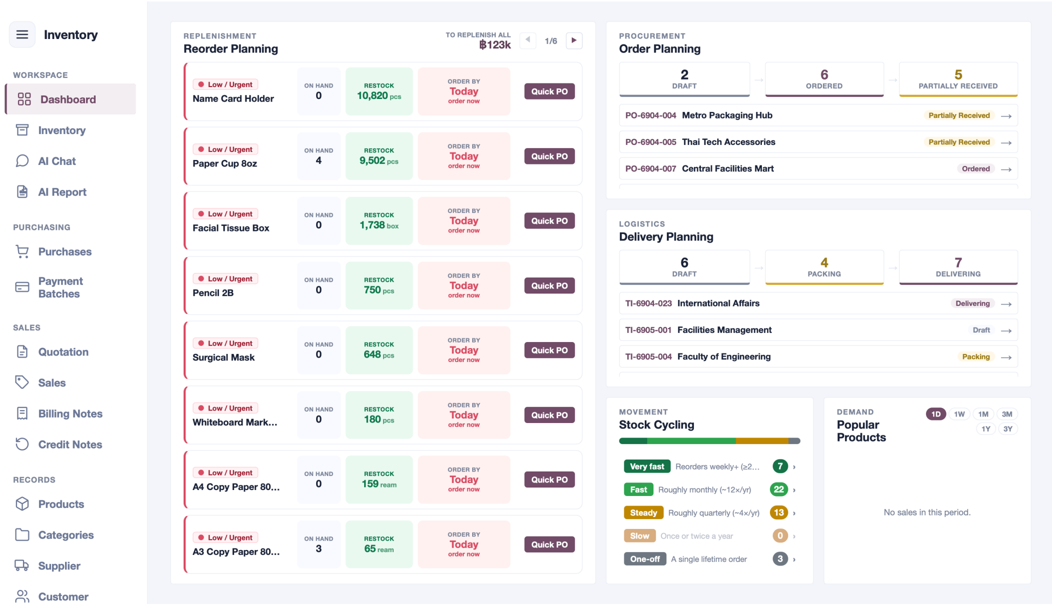

Figure 4.1 Dashboard Page

Figure 4.1 shows the dashboard page used as the main post-login business overview. The dashboard page was developed to receive backend dashboard data and show stock health, reorder planning, stock cycling, delivery pipeline, purchase planning, popular products, finance, cashflow, and order coverage. It is also connected to other pages, so a low-stock product can lead the user to the inventory or product page, and an order issue can lead to the purchase or sales page.

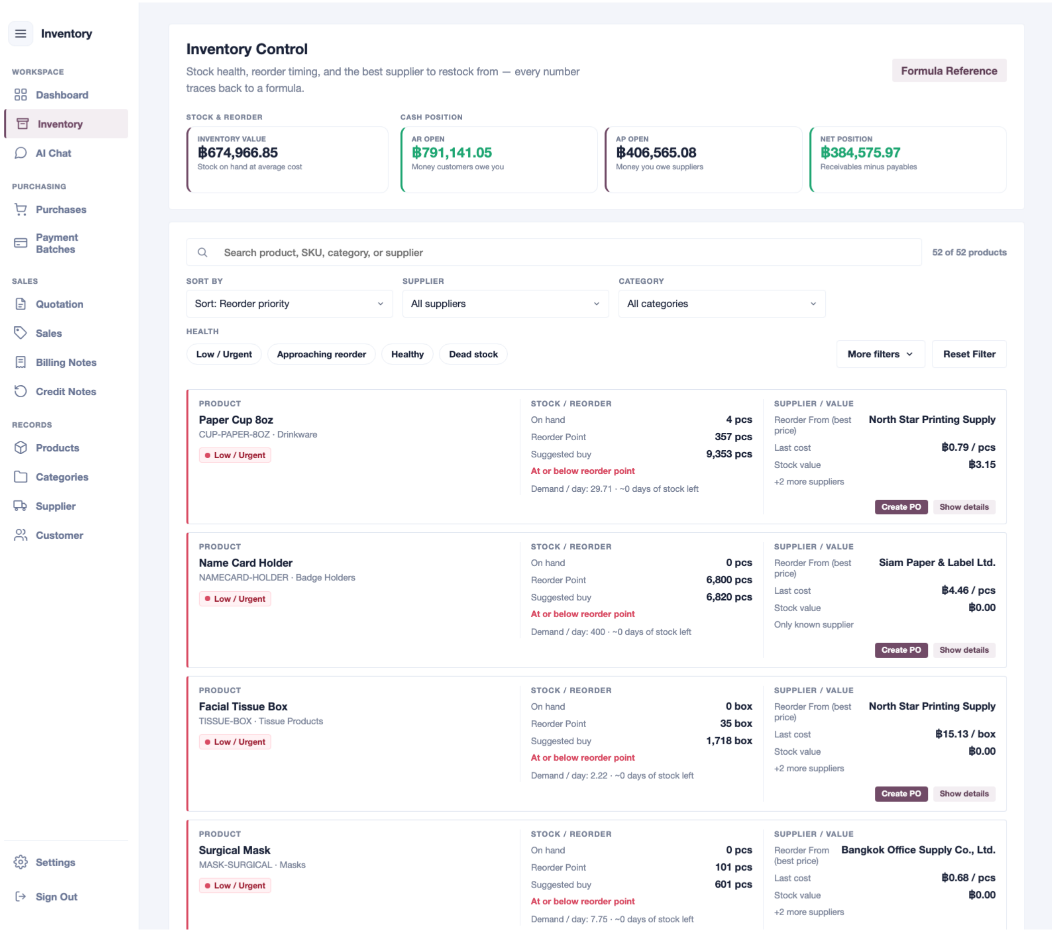

Figure 4.2 Inventory Page

Figure 4.2 shows the inventory page used for daily stock checking and reorder review. The inventory page was developed around backend stock report rows instead of calculating stock from full transaction lists in the browser. It includes an inventory control board, searchable directory, filters for health status, category, supplier, value, and reorder need, product detail views, and a quick purchase-order drawer.

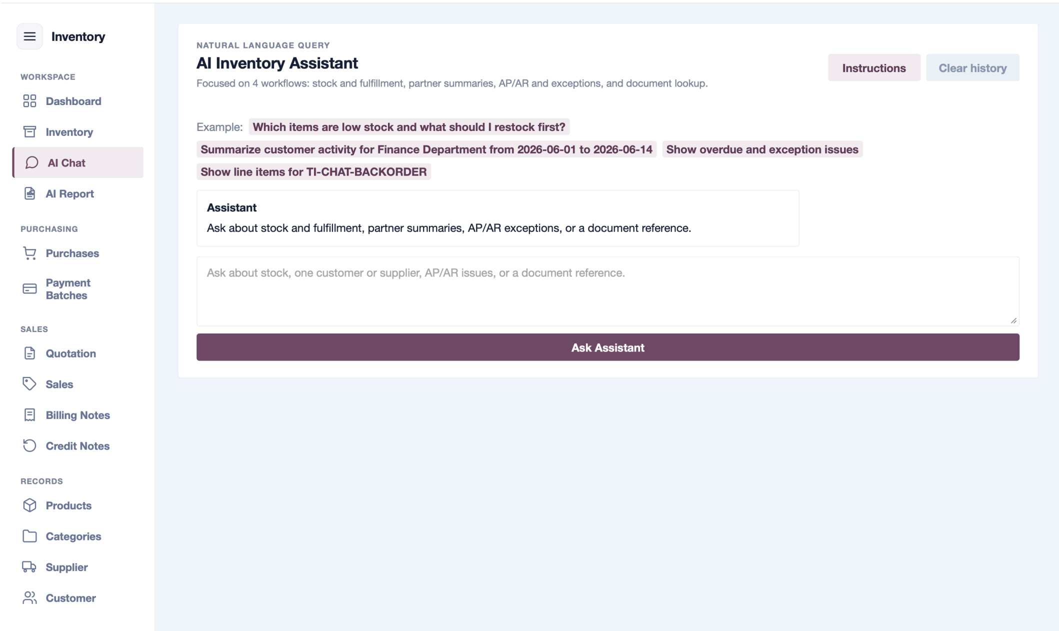

Figure 4.3 AI Chat Assistant Page

Figure 4.3 shows the AI Chat page used for read-only natural-language questions about stock, partners, receivables, payables, or document references. The backend first prepares the factual answer and structured presentation from current system data. If an OpenAI API key is configured, the model explains those verified facts and returns a separate conclusion and two to four question-specific highlights through a structured response. If the key is missing, the request fails, or the structured model output is invalid, the endpoint returns the deterministic local summary. The assistant does not create, edit, approve, cancel, or delete any record.

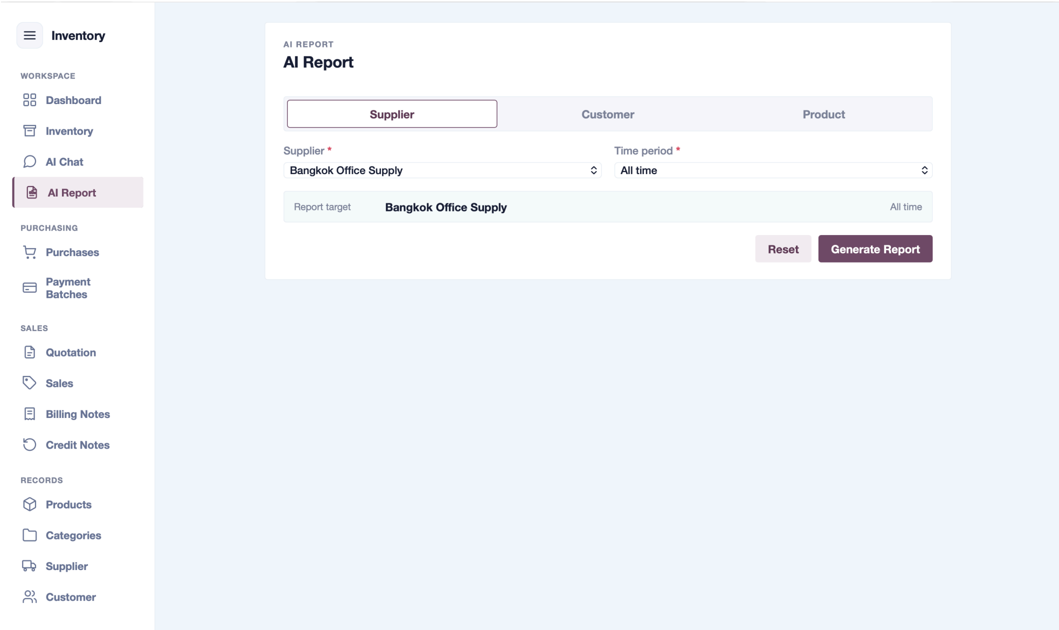

Figure 4.4 AI Report Generation Page

Figure 4.4 shows the AI report generation page for creating printable supplier, customer, or product reports. The user picks a scope (supplier, customer, or product), selects the specific record, and optionally narrows the date range. The frontend sends the request to the /api/ai-reports/ backend endpoint, where the backend collects related purchases, sales, quotations, billing notes, payment batches, credit notes, and stock data. If an OpenAI API key is configured, the backend sends the collected context to the AI model for an executive summary and business analysis; otherwise, it builds a local report from the same data. The final report opens in a new browser tab as a self-contained printable HTML document.

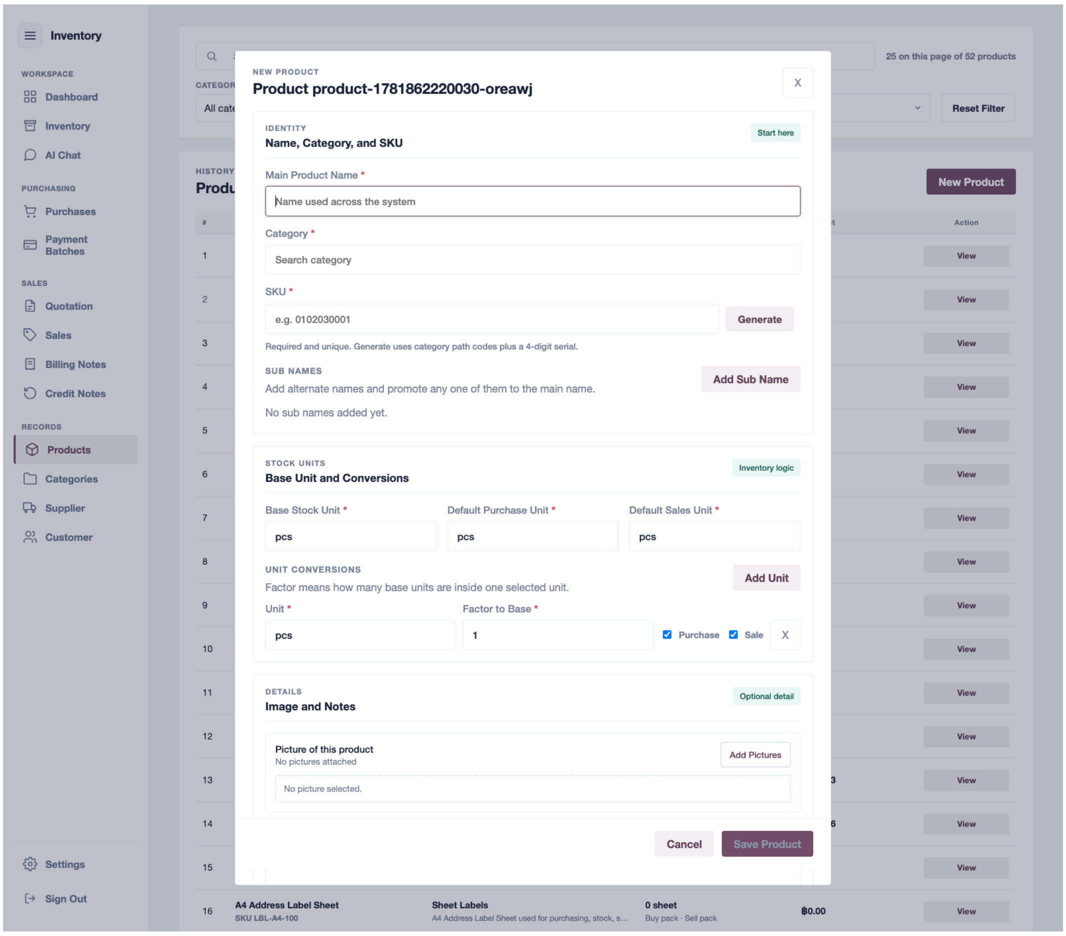

Figure 4.5 Product Management Form

Figure 4.5 shows the product management form for maintaining item master data. The Products page was developed as the main product catalog, including product creation, editing, disabling, deleting where allowed, SKUs, previous SKUs, names, category links, unit conversions, reorder levels, image or PDF attachments, supplier links, average cost and margin information, and product transaction history. Products with transaction history can be disabled instead of removed, which protects old purchase, sale, and quotation records.

The other master-data pages support the same business foundation. The categories page organizes products in a parent-child tree and shows the full hierarchy in one view. The supplier page stores company profile, contact details, taxpayer ID, branches, shipping addresses, remarks, payment terms, and procurement contact details. The customer page stores company profile, contact details, branches, shipping addresses, remarks, and billing or payment term information.

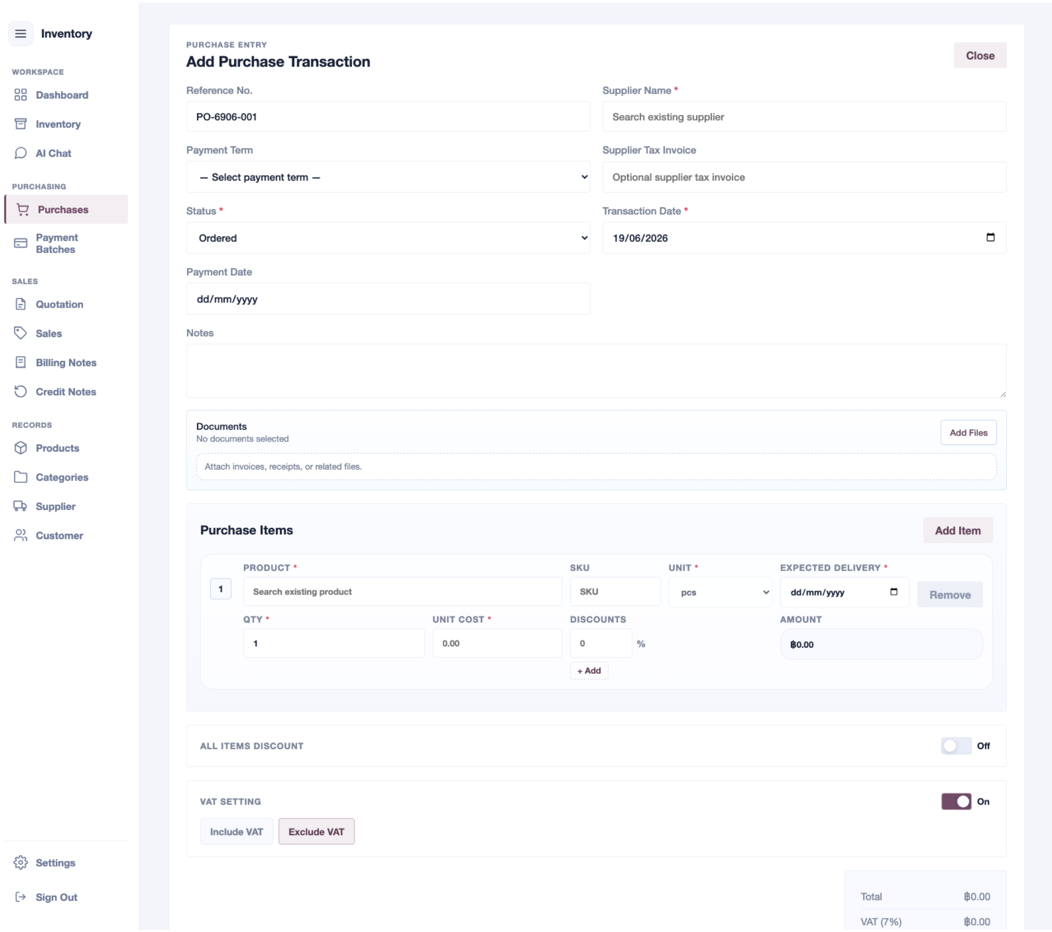

Figure 4.6 Purchase Workflow

Figure 4.6 shows the purchase workflow for entering and reviewing supplier purchase orders. The purchases page was developed for supplier-side transactions, including purchase records, line items, attached documents, purchase status, item receiving status, VAT mode, item discounts, bill discount, expected delivery dates, received dates, and payment batch links. Only received purchase items become available stock.

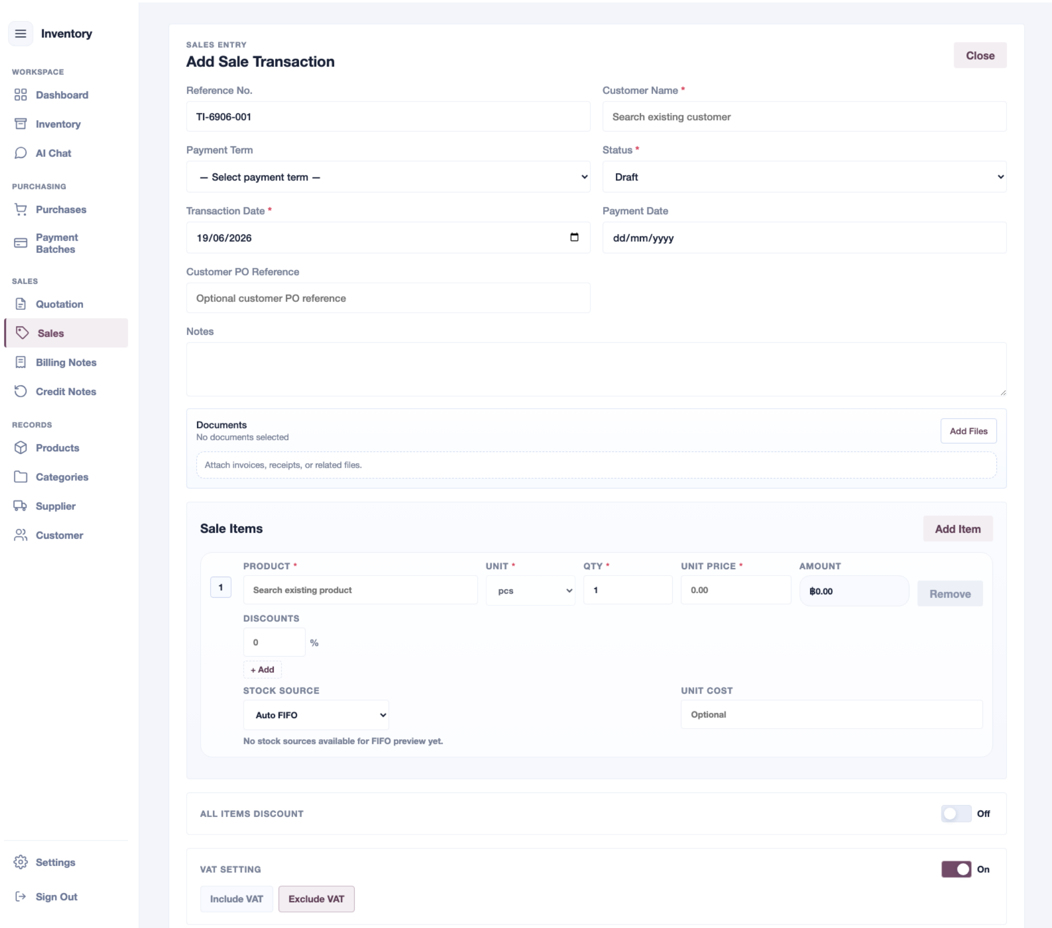

Figure 4.7 Sales Workflow

Figure 4.7 shows the sales workflow for preparing and tracking customer orders. The sales page was developed for customer-side transactions, including sale headers, customer information, customer purchase-order reference, sale items, documents, VAT totals, discounts, and delivery-related statuses. Sale item statuses include pending, packed, shipped, delivered, cancelled, and returned. The backend validates stock before a sale can commit stock, and when stock is committed, allocations are created from received purchase layers.

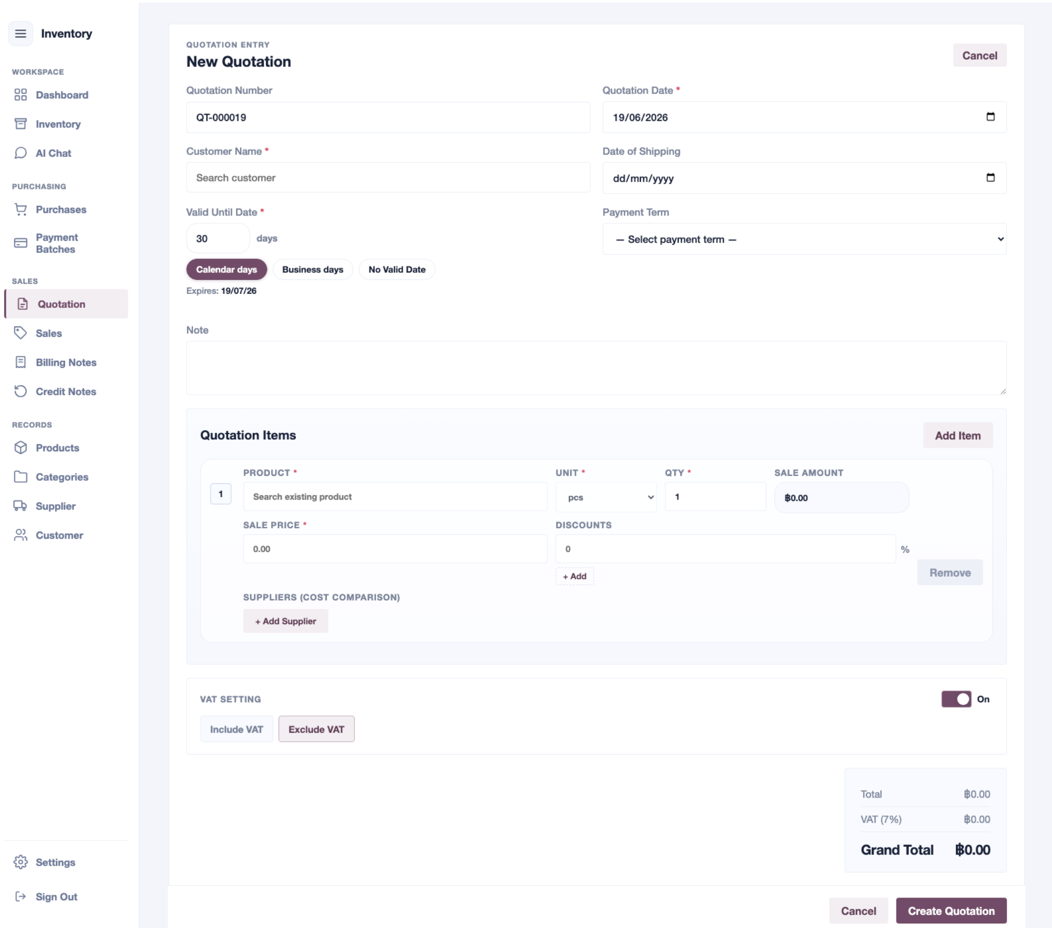

Figure 4.8 Quotation Workflow

Figure 4.8 shows the quotation workflow before a quote is converted into a purchase or sale. The Quotation page was developed for quotation line items, supplier cost options, selling prices, gross-margin review, validity settings, shipping date, payment terms, VAT mode, total values, and conversion to purchase or sales flow. In the database, quotation items are stored in the QuotationItem table and supplier options are stored in QuotationItemSupplier, while the API still returns an items array for frontend compatibility.

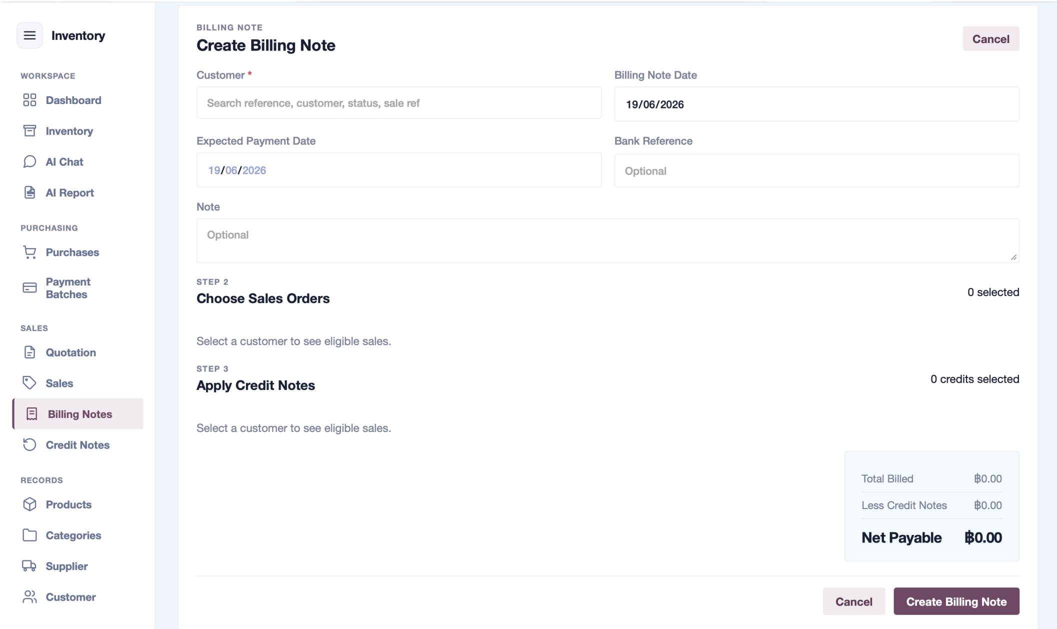

Figure 4.9 Billing Note Workflow

Figure 4.9 shows the billing note workflow for grouping eligible customer sales into receivable documents. The billing notes page was developed for customer receivable tracking. It loads backend-controlled eligible sales, groups them into billing notes, tracks whether each line has been received, and shows summary values, status filters, expected payment dates, actual payment dates, bank references, and linked credit note information.

The credit notes page was developed for adjustments after a sale is cancelled or returned. The backend provides eligible sale lines, and the credit note stores customer information, sale reference, optional billing note link, line items, and total credited amount. This keeps the credit document connected to the original sale item while preserving a readable adjustment record.

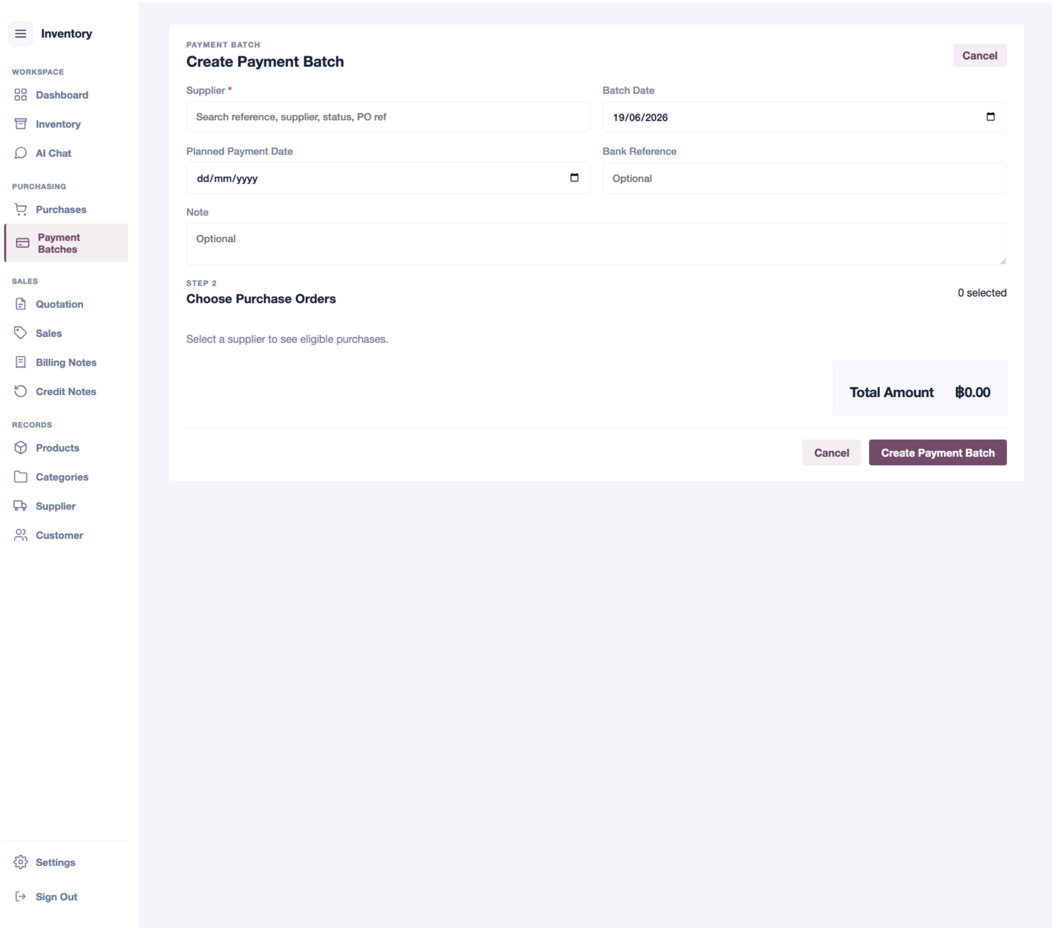

Figure 4.10 Payment Batch Workflow

Figure 4.10 shows the payment batch workflow for grouping eligible supplier purchases into payable batches. The payment batches page was developed to load eligible purchases from the backend instead of allowing users to manually select any purchase. Each payment batch stores supplier information, planned and actual payment dates, bank reference, total amount, and paid line status, keeping supplier payment tracking in one place.

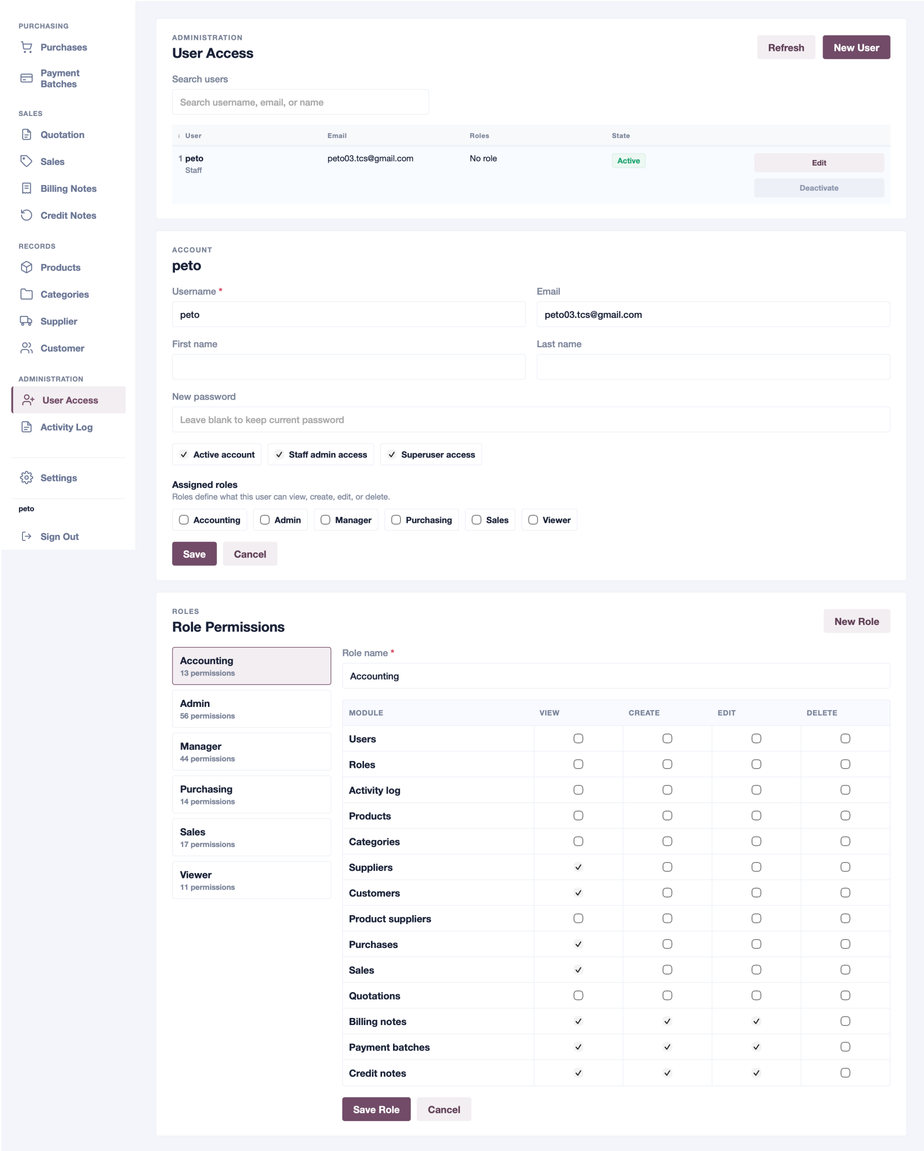

Figure 4.11 User Access Management Page

Figure 4.11 shows the user access management page for administrator account and role control. The login and access flow was developed with Simple JWT login, refresh, authenticated profile endpoints, frontend token storage, token refresh behavior, and logout handling. The backend-wide requirement for authentication can be controlled through the INVENTORY_REQUIRE_AUTH environment setting.

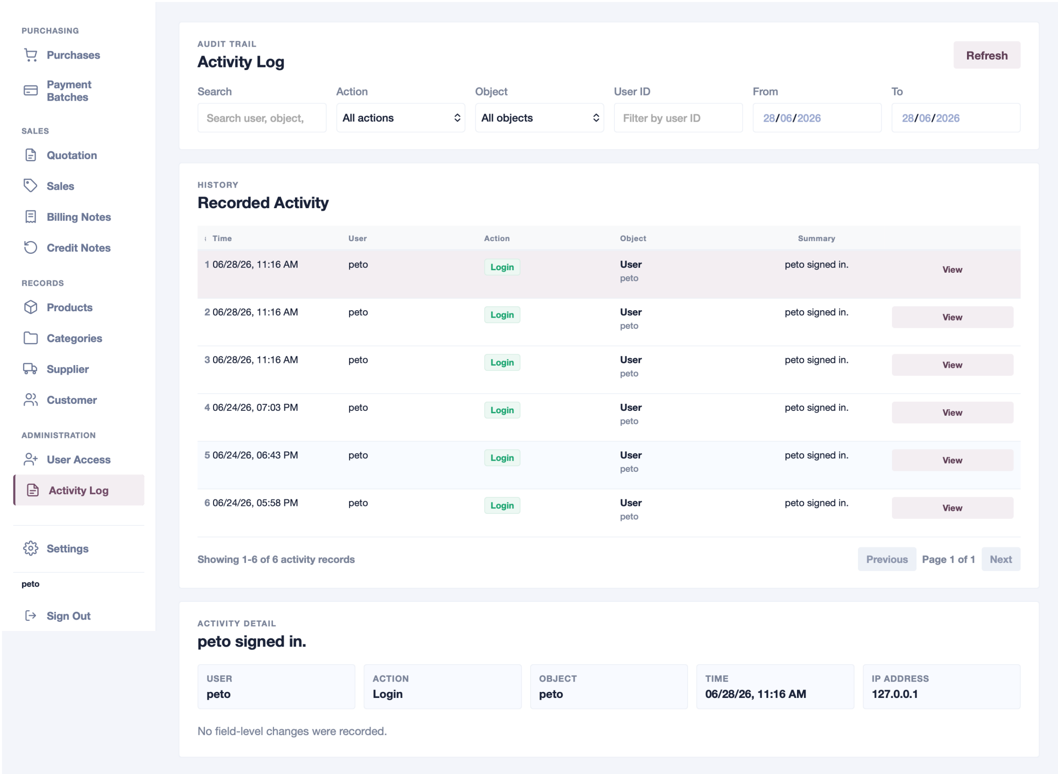

Figure 4.12 Activity Log Page

Figure 4.12 shows the activity log page for reviewing important system events such as login, create, update, and delete actions. The access system uses Django groups and permissions, with default roles such as Admin, Manager, Sales, Purchasing, Accounting, and Viewer to control which modules each user can view or update.

The Settings page completes the implemented navigation by allowing users to switch the interface between English and Thai. The language choice changes user-facing labels and guidance while preserving the same records and backend business rules.

4.4 Results by Workflow

The workflow results are connected directly to the relational database design. Master records such as products, suppliers, customers, and categories provide the reusable business entities. Transaction records then reference those master records while keeping snapshot fields, so historical documents remain readable even if master data changes later. Line-item tables store the actual product quantities, prices, statuses, and totals used by stock, finance, dashboard, and report calculations.

Table 4.1 Workflow Results and Related Database Entities

Workflow Area	Main Database Entities	Relationship to the Result
Master data	Product, Category, Supplier, Customer, ProductPicture, ProductUnitConversion, ProductSupplier	These tables store reusable product and partner records. Transactions reference them through foreign keys while also keeping snapshot names, SKUs, and terms for historical documents.
Purchasing and stock	Purchase, PurchaseItem, PurchaseDocument, Product, Supplier	Purchase headers link to suppliers, and purchase items link to products. Received purchase items are the inbound stock source used by inventory and dashboard calculations.
Sales	Sale, SaleItem, SaleItemAllocation, PurchaseItem, Customer, Product	Sale headers link to customers, sale items link to products, and allocation rows connect sold quantities back to received purchase layers for traceability.
Quotations	Quotation, QuotationItem, QuotationItemSupplier, Customer, Supplier, Product	Quotation headers store customer and supplier context, quotation items store requested products, and supplier option rows store alternative costs for each line.
Finance	BillingNote, BillingNoteLine, PaymentBatch, PaymentBatchLine, CreditNote, CreditNoteLine, Sale, Purchase	Finance documents group eligible sales or purchases through line tables, while credit notes reference the original sale so adjustments stay tied to the source transaction.
Dashboard and inventory	Product, PurchaseItem, SaleItem, ProductSupplier, BillingNote, PaymentBatch	Dashboard and inventory values are calculated from existing operational tables instead of storing a separate mutable stock total.
AI assistant and AI report	Product, Supplier, Customer, Purchase, Sale, Quotation, BillingNote, PaymentBatch, CreditNote	Assistant answers and printable reports use backend-prepared records from the same database entities that support the normal workflows.
User access and activity log	User, Group, Permission, ActivityLog	Django authentication tables store users, roles, and permissions, while the activity log records important system actions for review.

The key workflow result is that each module uses the same normalized database instead of keeping isolated screen-only data. Master data supports transactions, transaction line items drive stock and financial values, and finance documents reference eligible source records. This makes the system traceable from product setup through purchasing, sales, billing, payment, reporting, and audit review.

1. Master data provides the reusable product, category, supplier, and customer records needed by every transaction workflow.
2. Purchasing and sales line items are the main source for stock movement. Received purchase items add stock, while committed sale items allocate stock from received purchase layers.
3. Quotations, billing notes, payment batches, and credit notes keep business documents connected through database references, reducing duplicate or unrelated finance records.
4. Dashboard, inventory, AI chat, and AI report outputs use backend-prepared data from the same transaction tables, so summary information stays consistent with the stored records.
5. User access and activity logs add control and traceability by connecting staff accounts, roles, permissions, and important system events.

4.5 Feature Evidence Summary

Table 4.2 Evidence of Implemented Features

Feature	Evidence in the Implemented System
REST API communication	The frontend API client calls endpoints for dashboard, lookups, products, purchases, sales, quotations, billing notes, payment batches, credit notes, authentication, and chat.
JWT authentication and user access	The backend provides login, refresh, profile, user-management, role-management, and activity-log endpoints. The frontend stores and refreshes access tokens, shows permission-based navigation, and provides a user-access page for authorized administrators.
Backend stock validation	The sale workflow calls backend validation before stock-deducting sale statuses are committed.
FIFO and manual allocation	The backend creates sale allocations from received purchase layers and validates manual layer selections.
Finance eligibility	Backend endpoints return only eligible sales for billing notes, eligible purchases for payment batches, and eligible sale lines for credit notes.
Normalized quotation items	Quotation lines and supplier options are stored in relational tables while the API still returns a simple items array to the frontend.
Backend dashboard summaries	The dashboard and inventory pages use backend-prepared summary data instead of calculating all operational metrics from full transaction lists in the browser.
Assistant scope control	The chat service answers only supported read-only workflows and rejects unsupported requests as outside scope.
AI report generation	The AI report endpoint collects transaction data for a selected record, calculates metrics and charts, and returns a printable HTML report with optional AI-written analysis.

4.6 Testing and Evaluation Results

The system was evaluated using three types of evidence: automated backend validation, manual scenario testing, and user questionnaire results. Automated tests were used for business rules that must be exact, such as stock calculation, eligibility, and AI output validation. Scenario testing and questionnaires were used to evaluate how users performed the main workflow and how they perceived the system.

4.6.1 Automated Backend Validation

The final full backend suite contained 106 automated tests, and all 106 passed. This suite includes pagination, filtering, product data, transaction validation, stock allocation, finance eligibility, calculation, AI Chat, AI Report, user access, permission enforcement, and activity logging. For presentation continuity, the Blackbook also preserves a historical 30-case black-box aligned workflow selection; all 30 selected cases passed. The smaller recorded 5-test AI/calculation run and 6-test access/activity run are therefore treated as earlier focused evidence rather than the current total.

Table 4.3 Automated Validation Result Summary

Test Group	Result	Main Evidence
Full current backend suite	106/106 passed	The final Django inventory test suite passed across API, business-rule, calculation, AI, access-control, pagination, and filtering coverage.
Selected backend workflow tests	30/30 passed	Product rules, quotation behavior, purchase workflow, sale stock validation, finance eligibility, credit notes, payment batches, and payable synchronization passed.
Reorder point calculation	Passed	The backend returned reorder level 48, recommended restock 18, days until stockout 15, and stock value 300 from fixed mock data.
AI Chat validation	Passed	Local chat returned deterministic backend summaries; configured model responses retained the same presentation and added validated answer, conclusion, and highlights fields.
AI Report validation	Passed	Supplier, customer, and product reports used backend context, returned printable HTML, replaced poor model chart HTML with backend chart rows, and sanitized unsafe HTML.
User access and activity log validation	6/6 passed	Regular users were blocked from administrator endpoints, administrators could create users and assign roles, model permissions controlled protected access, and activity-log access was restricted.

4.6.2 Calculation Validation Detail

The calculation validation focused on the reorder point and recommended restock calculation because this is one of the most important inventory decisions in the system. The test did not use random data. Instead, it created fixed purchase and sale records in a temporary Django test database so that the expected result could be calculated manually.

The test first created one product, two received purchase records, one pending purchase record, two committed sales, and one pending sale. Then the test called the dashboard API, which runs the same backend stock report calculation used by the real dashboard and inventory page. The backend output was compared with the manually calculated expected values.

Table 4.4 Calculation Validation Input

Input Item	**Value**	**Reason for Test**
Product	TEST-PEN	A single controlled product makes the stock row easy to identify.
Received purchase units	100 pcs	Used as the stock added into the system.
Received purchase value	500	Used to calculate average unit cost.
Committed sale units	40 pcs	Used as stock already allocated to fulfilled sales.
Pending sale units	20 pcs	Used as future demand that should affect restock recommendation.
Pending purchase units	10 pcs	Used as incoming stock that should reduce restock recommendation.
Lead time samples	4 days and 6 days	Used to calculate average lead time of 5 days.
Sales history	40 pcs over 10 days	Used to calculate average daily demand of 4 pcs per day.

Table 4.5 Calculation Validation Output

Output Item	Expected Value	Backend Value	Result
Available stock	60	60	Passed
Average unit cost	5	5	Passed
Average lead time	5 days	5 days	Passed
Average daily demand	4 pcs/day	4 pcs/day	Passed
Safety stock	28	28	Passed
Reorder level	48	48	Passed
Recommended restock	18	18	Passed
Days until stockout	15 days	15 days	Passed
Stock value	300	300	Passed

The backend formula matched the manually expected values shown in Table 4.5.

4.6.3 AI Chat Validation Detail

The AI Chat validation checked whether the chat page answers from backend data instead of inventing business values. The test was performed by calling the backend chat function with fixed questions and known data. The important output fields were the answer text, presentation title, selected record references, model marker, conclusion, and highlights.

Without an API key, AI Chat returns a deterministic backend summary marked as local-summary. With a configured key, the model receives only the verified backend answer and presentation, then returns structured answer, conclusion, and highlight fields. If the model call or structured validation fails, the endpoint falls back to local-summary. Unsupported broad questions remain outside the assistant's operational scope.

Table 4.6 AI Chat Validation Input and Output

Test Question	Input Data Used	Expected Output Checked	Result
Customer activity summary	Customer with one known sale worth 275	Chat title shows the selected customer, sales count is 1, sales total is 275, and the sale record target points to the created sale.	Passed
Net position question	Existing billing note and payment batch data	Answer includes backend AR, AP, and net position values with presentation title ``Net position".	Passed
Low stock question	Product stock report rows from backend	Answer lists low-stock products and restock priorities from backend stock calculations.	Passed
Unsupported quotation summary	A broad request outside the supported assistant scope	Assistant returns ``Outside current assistant scope" and does not invent a quotation report.	Passed
Configured key condition	API key enabled with a mocked structured model response	Chat returns the configured model marker, model wording, conclusion, and highlights while retaining the backend presentation and record targets.	Passed

The validation records that both local and model-backed AI Chat answers use backend records and calculations as the factual source.

4.6.4 AI Report Validation Detail

The AI Report validation checked whether supplier, customer, and product reports are generated from the correct selected records. The test also checked that the final report is printable HTML, that chart rows come from backend data, and that unsafe model HTML is removed.

The report test used a controlled supplier and purchase record. The backend built a report context containing the supplier name, purchase reference, purchase amount, metrics, chart rows, and tables. With the API key setting configured, the model response was simulated so that the test could check the system behavior without depending on internet availability or changing AI output. This is important because automated tests must be repeatable.

Table 4.7 AI Report Validation Input

Input Item	**Value**	**Purpose**
Report scope	Supplier report	Confirms that the selected report type controls which records are collected.
Selected supplier	Configured Key Supplier	Confirms that the report context uses the selected supplier.
Product	Configured Key Product	Confirms that line-item data is included in the report context.
Purchase reference	PO-AI-KEY-001	Confirms that the report includes the expected transaction record.
Purchase amount	THB 450.00	Confirms that report totals and table values match backend data.
Model output	Simulated HTML body	Confirms how the system wraps model-style output into a final report document.

Table 4.8 AI Report Validation Output

Output Checked	Expected Result	Result
Report context	Contains Configured Key Supplier, PO-AI-KEY-001, and THB 450.00.	Passed
Printable document	Final HTML contains document structure and a print button.	Passed
AI model body handling	Simulated model summary remains in the final report after wrapping.	Passed
Chart rendering	Poor model chart labels are replaced with backend chart row labels.	Passed
HTML safety	Script tags, unsafe event handlers, iframes, and JavaScript links are removed.	Passed
Current-data report check	Supplier, customer, and product reports generated from current database records and contained expected references and totals.	Passed

The validation records that selected records, metrics, charts, and tables were prepared by the backend before report output.

4.6.5 Manual Scenario Test

The manual scenario test asked users to complete the main business flow from quotation to purchase, purchase receiving, payment batch, sale, credit note, and billing note. Ten testers finished the scenario test. Tester names were removed from the stored result file and replaced with anonymous labels for privacy.

Table 4.9 Scenario Test Participant Summary

Item	**Result**
Number of participants	10 testers
Age range	19 to 68 years old
Average age	43.5 years old
Roles represented	SME-related users and general evaluators, including owner, manager, purchasing/sales staff, executives, and students.
Scenario parts	Part A to Part H, covering quotation, purchase, receiving, payment, sale, credit note, and billing note workflow.

Table 4.10 Average Scenario Test Time and Score

Part	Average Time	Average Score	Interpretation
Part A	151.9 seconds	9.0/10	Initial quotation workflow took the longest because it starts the process and requires more data entry.
Part B	71.8 seconds	8.9/10	Purchase creation from quotation was generally successful, but some users needed time to understand the flow.
Part C	29.7 seconds	9.5/10	Receiving purchase items was one of the faster and clearer tasks.
Part D	65.1 seconds	9.5/10	Payment-related workflow received high scores.
Part E	40.4 seconds	9.2/10	Sale-related checking was successful, although some users needed to read more information.
Part F	43.1 seconds	8.9/10	Status-related actions worked, but comments showed that status buttons should be clearer.
Part G	7.2 seconds	9.3/10	The shortest part, showing that the required action was quick once found.
Part H	49.1 seconds	9.4/10	Final billing or finance-related workflow received high scores.

The average scenario score across all parts was 9.21 out of 10. The lower relative scores in Part B and Part F match written comments about quotation-to-purchase flow, sale form details, and status-button clarity.

4.6.6 Questionnaire Result

After the scenario test, testers answered a 15-item questionnaire using a 1 to 5 rating scale. The average questionnaire score across all testers was 4.58 out of 5.

Table 4.11 Questionnaire Result Summary

Evaluation Item	**Result**
Number of questionnaire responses	10 responses
Questionnaire items	15 items
Rating scale	1 to 5
Average questionnaire score	4.58/5
General result	Users gave mostly high scores, showing that the system was understandable and usable after completing the scenario.

4.6.7 User Feedback Summary

The user comments showed several repeated themes. Positive feedback mentioned that purchase order status was understandable, quotation-to-purchase flow was useful, credit note creation could be simple, and connected data reduced the need for repeated entry. Improvement feedback focused mainly on visual clarity and consistency.

Table 4.12 User Feedback Themes

Theme	Summary of Feedback
Easy areas	Users found purchase status, quotation creation, quotation-to-purchase flow, and connected data flow useful and understandable.
Difficult areas	Some users had difficulty finding buttons, especially save buttons, detail buttons, status buttons, and finance-related options such as billing notes or credit notes.
Visual clarity	Several comments suggested stronger colors, clearer status indicators, and more obvious emphasis for important actions or warning values.
Workflow consistency	Some users felt that clickable rows and save-button placement should be more consistent across pages.
Information density	Some users felt that certain pages, such as dashboard charts or cost information sections, contained many details and should be simplified or visually reorganized.

The recorded feedback identifies the main improvement areas as button visibility, status clarity, finance-action discoverability, and information density.

4.7 Limitations Found During Development

The system is functional, but some limitations remain:

- Backend-wide authentication is configurable, so the final deployment setting must be checked before production use.
- User access is implemented with role and permission management, but production deployment should still review the final staff roles, permission assignments, and administrator accounts before real company use.
- The frontend still keeps mock-data fallback behavior for demonstration, so final evaluation should use backend-connected data.
- Large-scale production performance testing with many users and a very large database was not included in the current codebase.

- User testing was conducted with 10 participants, so the result is useful for project evaluation but not enough to represent all possible SME users.

CHAPTER 5
CONCLUSION

5.1 Brief Project Overview

The Inventory Management System was developed as a web application for SME trading businesses that buy products from suppliers and resell them to customers. The system covers product and partner records, purchases, sales, quotations, billing notes, payment batches, credit notes, inventory and margin review, dashboard summaries, limited AI-assisted questions, printable AI-generated reports, administration, activity logs, and language settings.

The project was built with a React and Vite frontend, a Django REST Framework backend, and a MySQL database. The main idea of the system is to connect stock, documents, and finance records together so the user can follow the business workflow from purchase to sale and payment.

5.2 Objective Summary

The project objectives focused on three main goals: recording purchase and sales transactions with status and document attachment support, showing useful inventory and margin information, and adding an AI chatbot that can answer supported questions from backend data.

The implemented system meets these goals within the current project scope. Purchases and sales support status tracking and documents. Inventory stock is calculated from received purchases and committed sales, while cost and selling-price data support gross-margin review. The assistant can answer supported read-only questions but cannot create or edit records; configured AI wording remains grounded in the backend-prepared presentation.

5.3 System Design Summary

The system was designed as a three-layer web application. The frontend handles the pages and user interaction. The backend handles APIs, validation, stock rules, eligibility rules, dashboard summaries, and assistant context. The database stores the actual business records using relational tables.

This design was useful because it keeps important business rules on the backend. The frontend can make the system easier to use, but the backend still controls the rules that protect stock, billing, supplier payments, and credit notes.

5.4 Implementation Summary

The frontend implementation includes pages for Dashboard, Inventory, AI Chat, AI Report, Purchases, Payment Batches, Quotation, Sales, Billing Notes, Credit Notes, Products, Categories, Supplier, Customer, User Access, Activity Log, and Settings. It also includes a shared API client, authentication handling, permission-based navigation, pagination support, filters, modals, document references, bilingual text, and mock-data fallback for development.

The backend implementation includes Django models, serializers, viewsets, service functions, dashboard endpoints, lookup endpoints, eligibility endpoints, authentication endpoints, user and role management endpoints, activity logging, AI report generation, management commands, and backend tests. The most important backend logic includes stock calculation, stock allocation, sale stock validation, margin inputs, purchase payable calculation, finance eligibility, permission enforcement, assistant fact preparation, structured AI response validation, and AI report context building.

5.5 Result Summary

The final result is a working inventory management system that connects the main trading workflows. A received purchase can increase stock, a committed sale can reduce stock, a billing note can track money expected from a customer, and a payment batch can track money that needs to be paid to a supplier.

The dashboard and inventory pages make the system easier to review because they summarize stock and finance information in one place. The assistant adds another way to ask about supported operational information while staying within a narrow read-only scope. When configured, OpenAI adds an explanation, conclusion, and highlights to verified backend facts; local fallback remains available. The AI Report page generates printable supplier, customer, or product reports with metrics, charts, and business analysis.

The quantitative evaluation was consistently positive. The final automated backend suite passed 106 of 106 tests, while the preserved presentation subset passed 30 of 30 cases. Ten scenario-test participants achieved an average score of 9.21 out of 10, and the questionnaire average was 4.58 out of 5. These results show strong technical correctness and favorable user evaluation within the tested scope.

5.6 Key Findings

The main finding is that the system is most useful when inventory, margin, sales, purchasing, billing, payment, and reporting are connected through the same transaction records. This makes stock status, reseller margin, financial follow-up, and business documents easier to check from one system instead of being managed separately. The evaluation also indicates that the main remaining usability issues are button visibility, status clarity, finance-action discoverability, and information density rather than missing core workflows.

5.7 Obstacles and Challenges

The main challenge was keeping connected workflows consistent. Purchase, sale, stock, billing, payment, and credit-note records affect one another, so the backend had to enforce rules while the interface still stayed practical for daily users.

5.8 Future Work

Future work should focus on production readiness and usability:

1. Test the system with a larger SME user group and larger operational database.
2. Review production roles, permissions, and deployment settings with a real busi-

ness.

3. Improve audit details and visual clarity for important workflow status changes.

REFERENCES

[1] K. Bishop. Qtier-Rapor: Managing spreadsheet systems and improving corporate performance, compliance and governance, 2008. URL `https://arxiv.org/abs/0803.0011`.

[2] Y. Lappalainen and N. Narayanan. Aisha: A custom ai library chatbot using the ChatGPT api. *Journal of Web Librarianship*, 17(3):37--58, 2023. doi: 10.1080/19322909.2023.2221477.

[3] H. Mittal and M. Arora. A Full-Stack Web Solution for Online FMCG (Fast-Moving Consumer Goods) Management System Using MERN: Design, Implementation and Evaluation. *International Journal of Latest Technology in Engineering, Management & Applied Science*, 14(4):1089--1094, 2025. doi: 10.51583/IJLTEMAS.2025.140400132.

[4] M. Muller. *Essentials of Inventory Management*. AMACOM, New York, NY, 2 edition, 2011. ISBN 9780814416556.

[5] Office of Management and Budget. *North American Industry Classification System: United States, 2022*. Executive Office of the President, Office of Management and Budget, 2022. URL `https://www.census.gov/naics/reference_files_tools/2022_NAICS_Manual.pdf`. Sector 42, Wholesale Trade.

[6] OpenAI. *Structured Model Outputs*. OpenAI Developer Documentation, 2026. URL `https://developers.openai.com/api/docs/guides/structured-outputs`. Accessed 14 July 2026.

[7] E. A. Silver, D. F. Pyke, and D. J. Thomas. *Inventory and Production Management in Supply Chains*. CRC Press, Boca Raton, FL, 4 edition, 2016. ISBN 9781466558618.

[8] The Times. Sole traders still not prepared for digital tax returns in april. The Times, December 2025. Reports Sage research on sole traders' use of spreadsheets, bank statements, and pen-and-paper accounting.

APPENDICES

APPENDIX A.
SOFTWARE USER GUIDE

This appendix provides a practical guide for using the Inventory Management web application. It is written for daily users such as administrators, product controllers, purchasing staff, sales staff, and finance staff. The guide follows the same business workflow used by the system: prepare master data, create quotations, buy from suppliers, sell to customers, follow up billing and payments, and review the business situation.

A.1 Starting the Software

The software can be started in two main ways. The recommended approach for demonstration is Docker Compose because it starts the MySQL database, Django backend API, and React/Vite frontend together. Manual startup can also be used when the developer wants to run each service directly on the local machine.

A.1.1 Starting with Docker Compose

Before starting, Docker Desktop should be installed and running. From the project root directory, run the following command:

```
docker compose up --build
```

The first startup may take a few minutes because Docker needs to download images and build the application containers. When startup is complete, open the frontend application in a browser:

http://127.0.0.1:5173/

The backend API and Django admin are available at:

http://127.0.0.1:8000/api/
http://127.0.0.1:8000/admin/

The backend container runs database migrations automatically before the server starts. The MySQL container uses port 3306 inside Docker and is exposed to the local machine on port 3307 to avoid conflict with a local MySQL server.

To add demonstration operational data after the containers are running, use:

```
docker compose exec backend python manage.py seed_operational_data
```

To create an administrator account for the Django admin page, use:

```
docker compose exec backend python manage.py createsuperuser
```

To stop the running containers while keeping the database volume, use:

```
docker compose down
```

To reset the Docker database completely, use the following commands only when the existing Docker database data should be deleted:

```
docker compose down -v
docker compose up --build
```

If AI Chat and AI Report should use live AI generation, start Docker Compose with an OpenAI API key:

```
OPENAI_API_KEY=your-key docker compose up --build
```

A.1.2 Starting Manually without Docker

Manual startup requires a local MySQL server, Python, and Node.js. First, create the MySQL database and user expected by the default backend environment file:

```
CREATE DATABASE inventory_db
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

```
CREATE USER 'inventory_user'@'localhost'
IDENTIFIED BY 'inventory_password';
GRANT ALL PRIVILEGES ON inventory_db.* TO 'inventory_user'@'localhost';
GRANT ALL PRIVILEGES ON test_inventory_db.* TO 'inventory_user'@'localhost';
FLUSH PRIVILEGES;
```

In the first terminal, start the backend:

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python manage.py migrate
python manage.py runserver 127.0.0.1:8000
```

In the second terminal, start the frontend:

```
cd frontend
npm install
npm run dev -- --host 127.0.0.1
```

Then open the application at:

```
http://127.0.0.1:5173/
```

With the backend virtual environment active inside the backend directory, demonstration data and an administrator account can be created with:

```
python manage.py seed_operational_data
python manage.py createsuperuser
```

A.2 Accessing the System

Users access the system through the web application sign-in page. A staff member should sign in with the username and password assigned by the administrator. If the system is used for demonstration or training only, the guest mode can be used to open the application with mock data. Guest mode should not be used for real business records because changes are not saved to the live database.

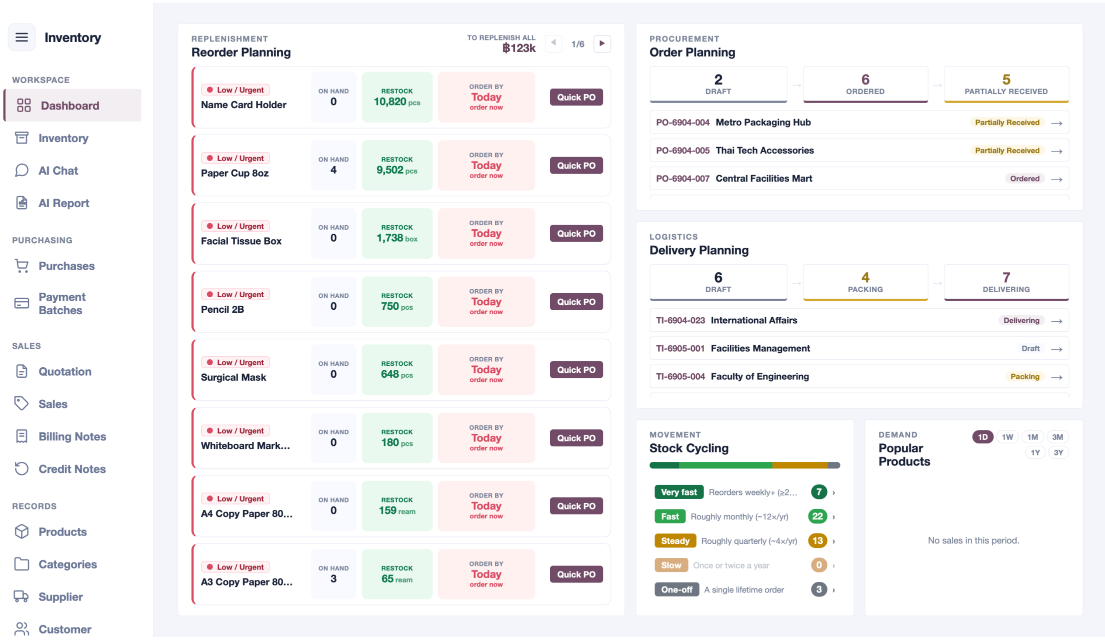

Figure A.1 Main Sidebar Navigation

After signing in, the sidebar is used to move between the main modules. The workspace group contains Dashboard, Inventory, AI Chat, and AI Report. The purchasing group contains Purchases and Payment Batches. The sales group contains Quotation, Sales, Billing Notes, and Credit Notes. The records group contains Products, Categories, Supplier, and Customer. Administration pages such as User Access and Activity Log are shown only to users with suitable permissions, while Settings provides the English/Thai language choice.

A.3 Basic Operating Rules

Before using the system, users should understand the following rules because they affect stock, cost, and financial records.

1. Product stock is calculated from transactions. Users do not type a free stock

number into the product record.

2. Purchase items increase available stock only after the item status is changed to received.
3. Sales items reserve or consume stock when they move into stock-affecting statuses such as packed, shipped, or delivered.
4. The system uses first-in, first-out allocation by default, but users may choose manual stock layers when a specific received batch is physically used.
5. Historical transactions keep snapshot information such as product names, partner names, prices, tax values, and totals. Later edits to master data should not rewrite old business documents.
6. Billing notes, payment batches, and credit notes should be created from eligible records shown by the system instead of manually choosing unrelated transactions.

A.4 First-Time Master Data Setup

The first step is to prepare the records that transactions depend on. Categories should be created as a clear hierarchy. Products should then be linked to the correct category and configured with SKU, base unit, purchase unit, sales unit, unit conversions, and reorder information. Suppliers and customers should also be created before purchase and sales transactions are entered.

Table A.1 Master Data Setup Checklist

Module	**Required Setup**
Categories	Create the parent and child category tree used to organize products.
Products	Enter SKU, product name, category, base unit, purchase and sales units, unit conversions, reorder point, active status, and any image or PDF attachment.
Suppliers	Enter supplier name, tax ID, contacts, address, payment terms, and bank or procurement details.
Customers	Enter customer name, tax ID, billing address, shipping address, contacts, and payment terms.

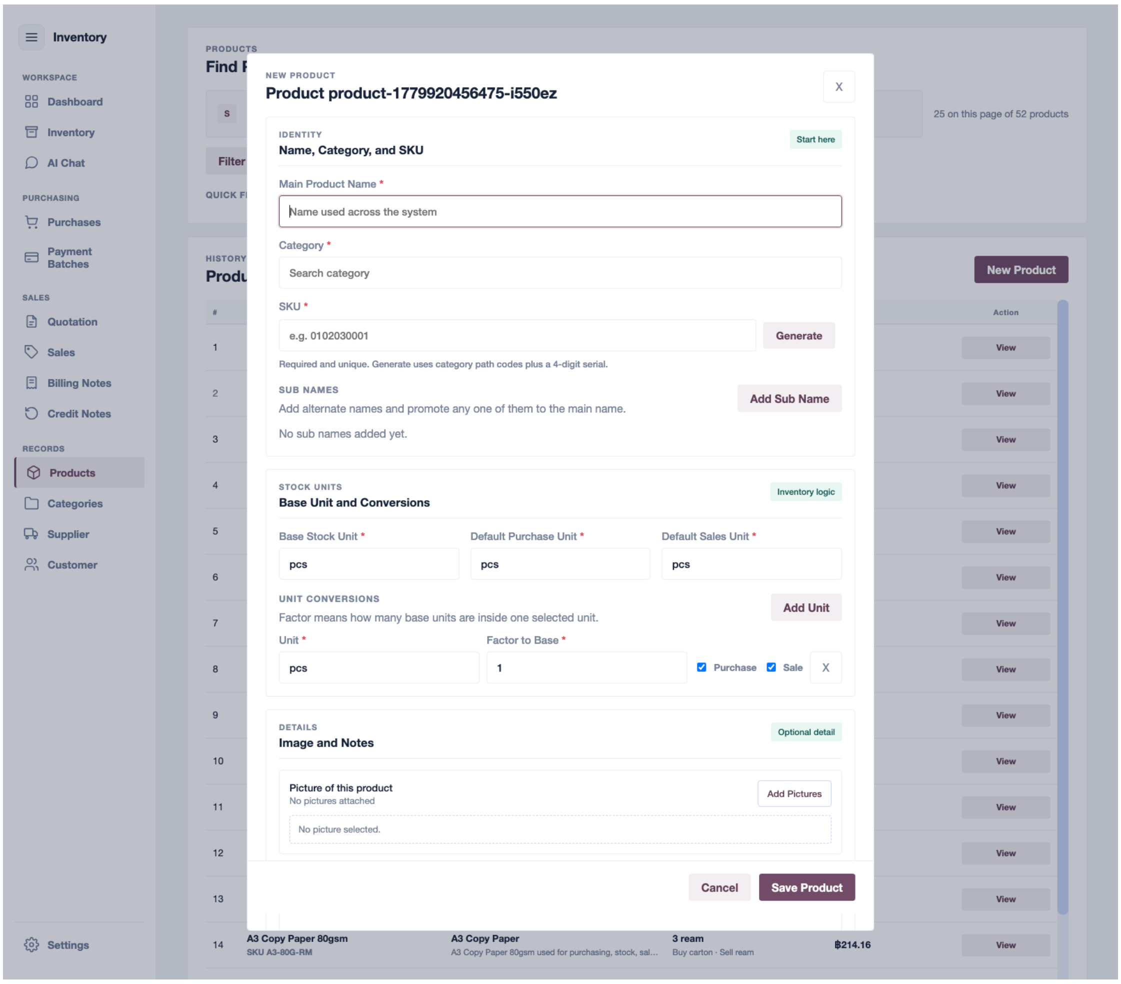

Figure A.2 Product Form With Unit Conversion Setup

The base unit is especially important. It should represent the smallest unit used for stock calculation. For example, if a product is bought by box but sold by piece, the base unit may be piece and the purchase unit may be box. A conversion such as one box equals ten pieces allows the system to convert purchase quantities into stock quantities correctly.

A.5 Daily Transaction Workflow

The normal daily workflow begins with a quotation or a direct purchase or sale. A quotation records customer demand and checks whether the requested items have enough available stock. If the quotation shows that stock is short, the user can prepare a purchase from the short-stock lines. If stock is available, the user can convert the quotation into a sale.

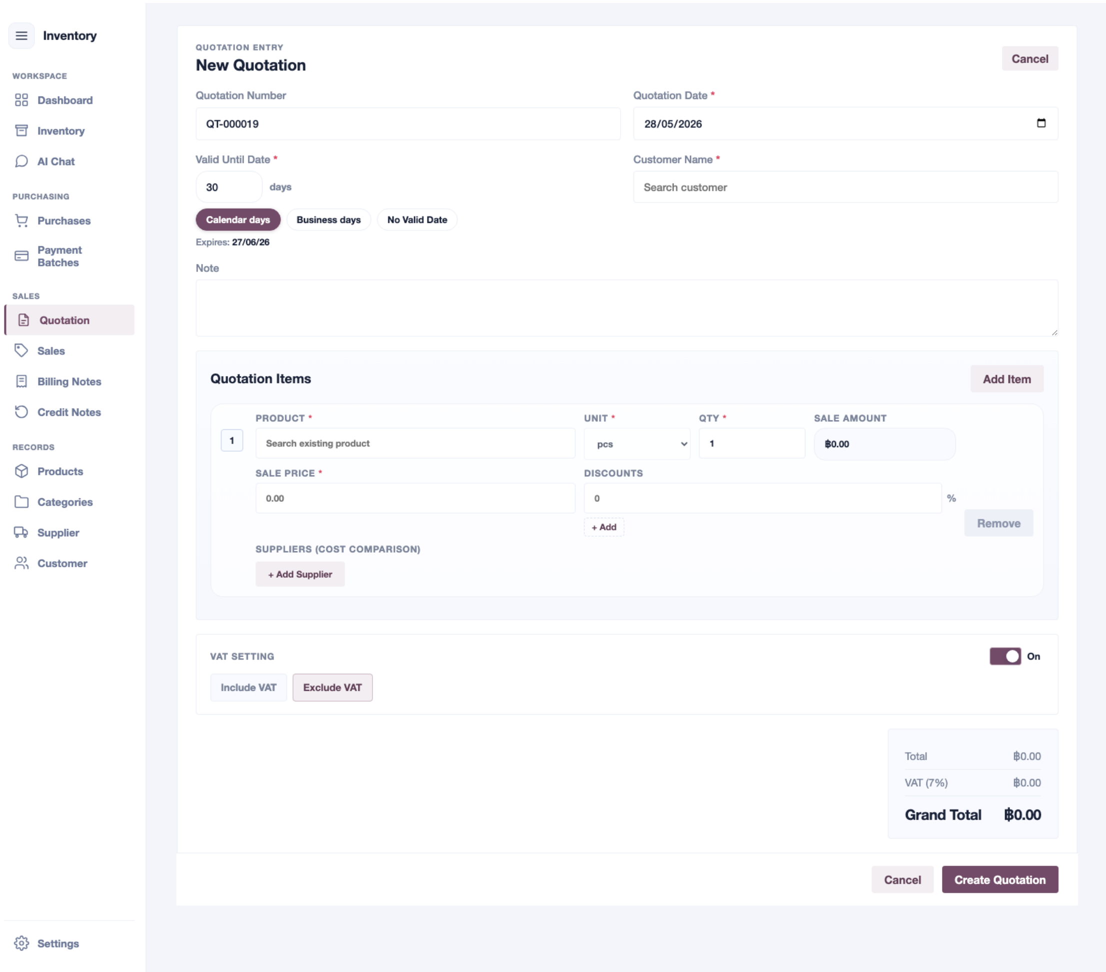

Figure A.3 Quotation Entry With Stock Sufficiency Information

Purchases record supplier-side orders. Users choose the supplier, add products, quantities, units, costs, and expected delivery dates, then save the purchase. When goods arrive, each purchase item should be updated to received only after the physical stock has been checked. This receiving action is what makes stock available for future sales.

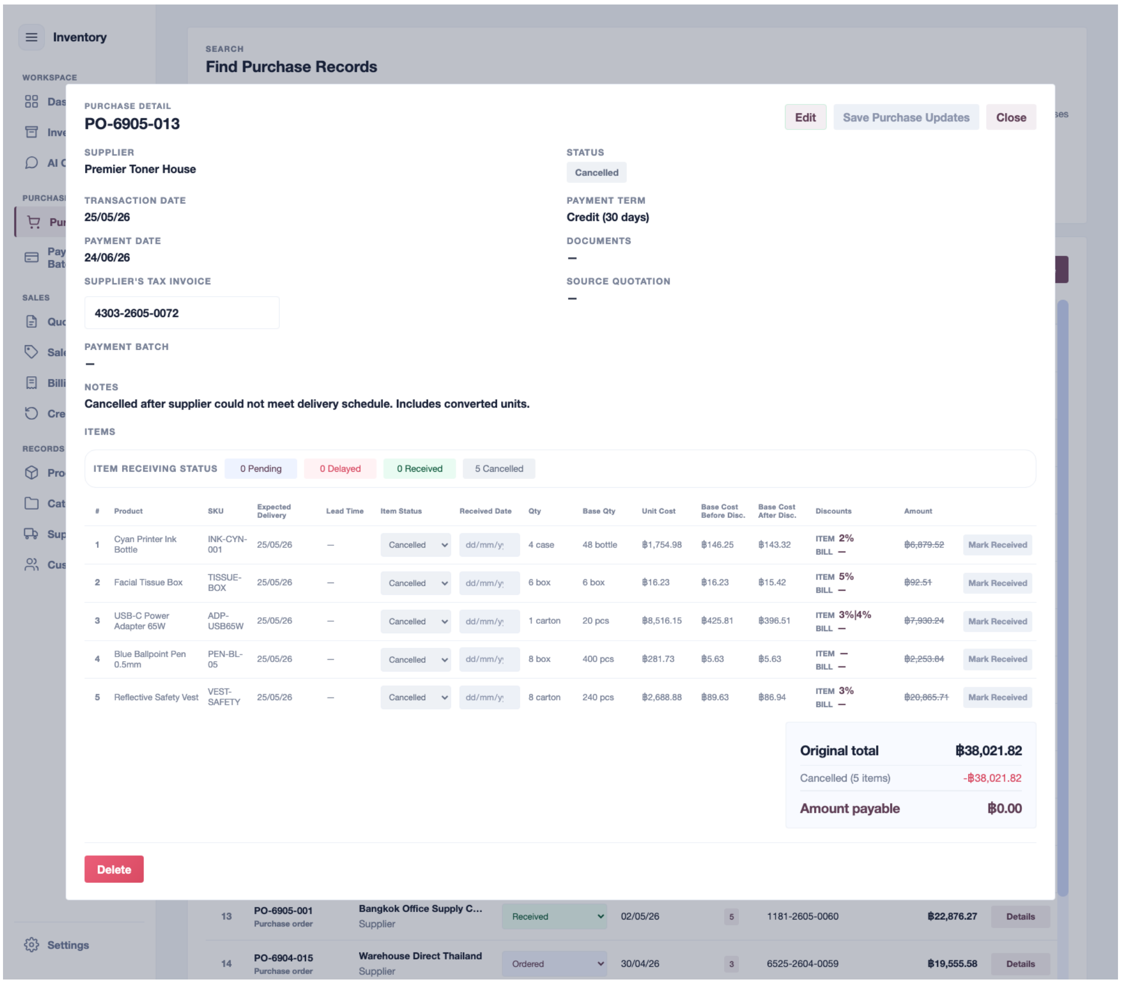

Figure A.4 Purchase Detail With Receiving Status

Sales record customer-side orders. Users choose the customer, add products, quantities, prices, and delivery-related information. The system checks stock and creates stock allocations from received purchase layers. If the user does not choose a specific layer, the system allocates stock using FIFO. Sales items should only be marked packed, shipped, delivered, cancelled, or returned when the real operational event has occurred.

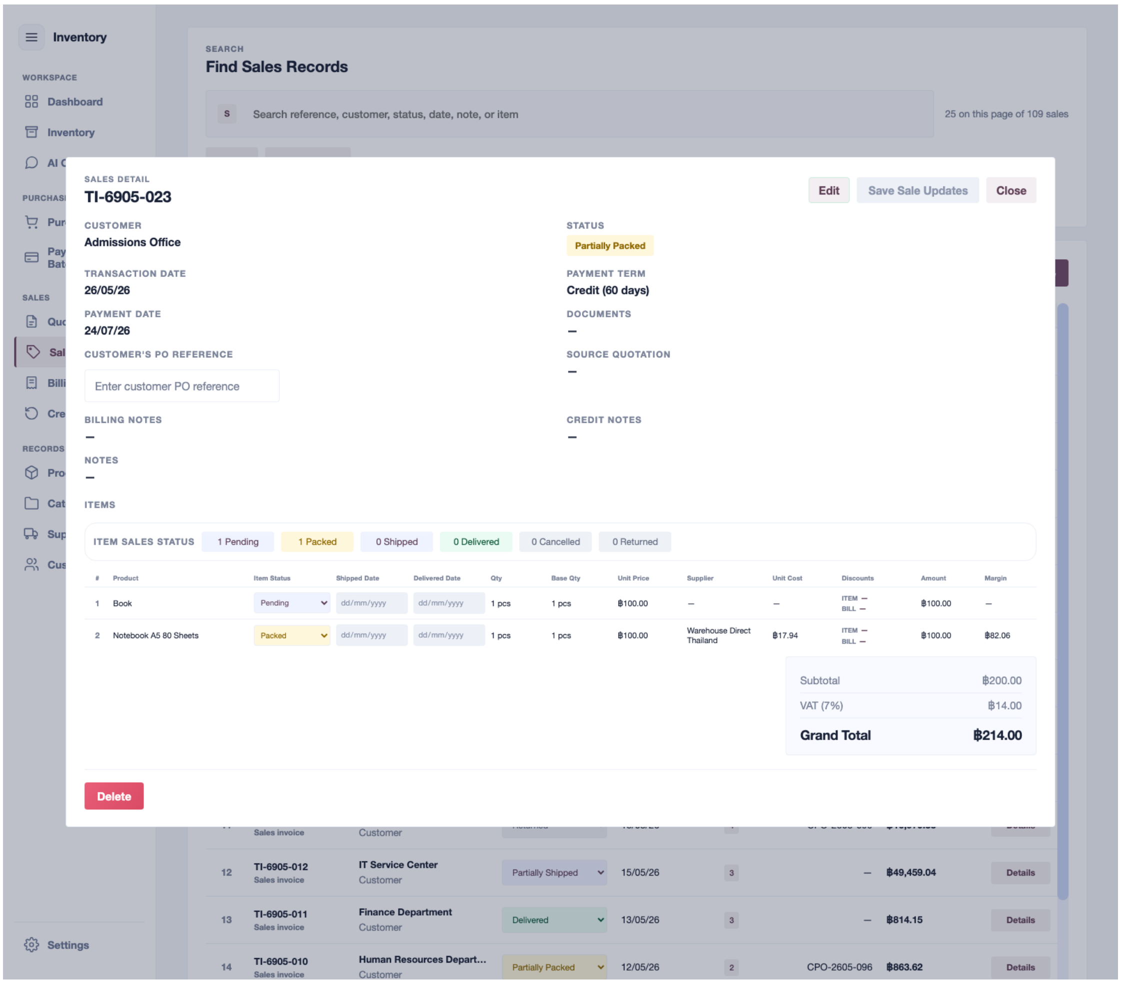

Figure A.5 Sales Detail With Delivery Status

A.6 Finance Follow-Up Workflow

Finance users use billing notes, payment batches, and credit notes to follow up completed or adjusted transactions. Billing notes group eligible customer sales for collection. Payment batches group eligible supplier purchases for payment. Credit notes record returned or cancelled sale items.

Table A.2 Finance Follow-Up Modules

Module	**Business Side**	**Main User Action**
Billing Notes	Customer receivable	Select eligible shipped or delivered sales, issue the billing note, and record payment received.
Payment Batches	Supplier payable	Select eligible received purchases, schedule payment, and record the paid status and bank reference.
Credit Notes	Customer adjustment	Create credit notes from cancelled or returned sale items and review the tax and value adjustment.

These modules rely on backend eligibility checks. If a sale or purchase does not appear in the eligible list, the user should check its status, partner, and whether it has already been attached to another active document.

A.7 Inventory and Business Review

The dashboard and inventory pages are used for daily review. The dashboard summarizes urgent reorder items, stock movement, delivery planning, cash-flow pressure, popular products, and order coverage. The inventory page gives a more detailed stock-control view with product stock, pending purchases, demand, reorder values, supplier options, and product history.

Users should review low-stock products before promising new delivery dates. They should also check delayed inbound purchases because a late supplier delivery can affect customer sales. Product history can be opened when a user needs to audit the purchase and sales records behind a stock value.

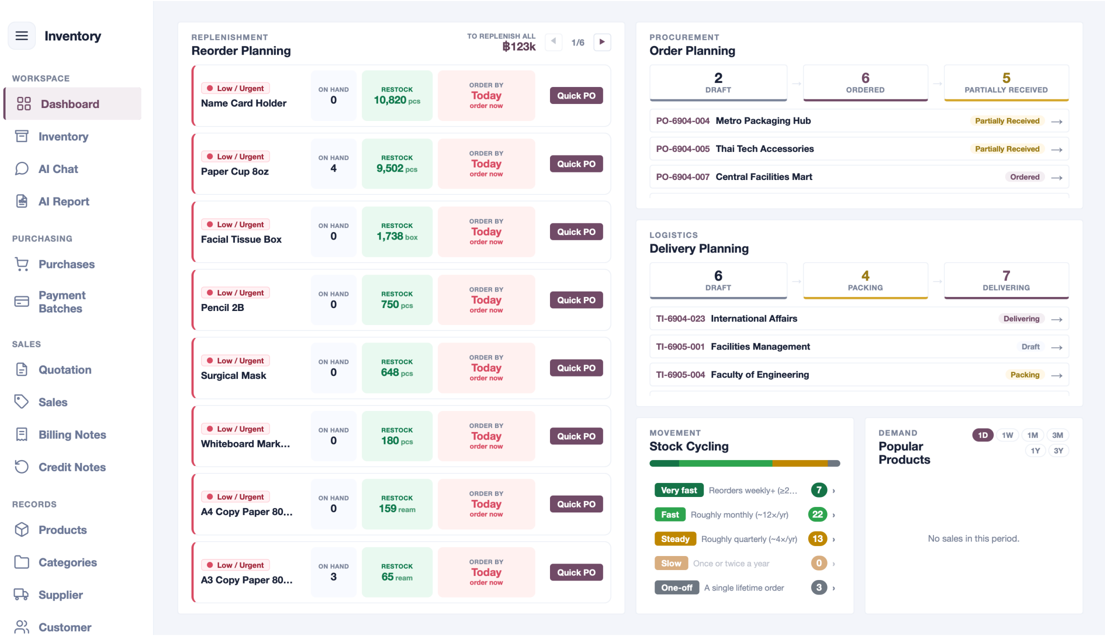

Figure A.6 Dashboard Overview for Business Review

A.8 AI Chat and AI Report

AI Chat is a read-only helper for asking supported operational questions. Suitable questions include low-stock checks, supplier or customer summaries, overdue receivables, overdue payables, order coverage, and document reference lookups. With an OpenAI API key configured, the answer includes AI-written wording, a conclusion, and question-specific highlights based on the verified backend presentation. Without a key or after an API failure, the page displays the deterministic local summary. In both modes, backend data and transaction records remain the source of truth.

AI Report creates printable reports for a selected supplier, customer, or product. The user selects the report scope, selects the record, chooses a reporting period if needed, and generates the report. The generated report includes backend-prepared metrics, related records, chart data, and a written analysis. The report can be printed or saved as a PDF from the browser.

A.9 Administration

Administrators use the User Access page to create staff accounts, assign roles, activate or deactivate users, and review role permissions. The Activity Log page is used to review important system activity such as login, creation, update, and deletion events.

Staff should be assigned only the access required for their responsibility.

A.10 Quick Reference

Table A.3 Quick Reference for Common User Goals

Goal	Page	Main Action
Check business overview	Dashboard	Review stock, delivery, receivable, payable, and coverage summaries.
Check stock and reorder needs	Inventory	Filter stock rows, inspect reorder signals, and open product history.
Create product records	Products	Set SKU, category, base unit, unit conversions, suppliers, and reorder information.
Prepare a customer offer	Quotation	Add requested items, review stock sufficiency, and convert to purchase or sale when appropriate.
Buy from supplier	Purchases	Create purchase orders and mark items received only after goods arrive.
Fulfill customer order	Sales	Create sales, review FIFO allocation, and update delivery statuses.
Collect customer payment	Billing Notes	Group eligible sales and record received payment.
Pay supplier	Payment Batches	Group eligible purchases and record paid status.
Record returns or cancellations	Credit Notes	Issue credit notes from returned or cancelled sale items.
Ask operational questions	AI Chat	Ask supported read-only questions about stock, partners, AP, AR, and documents.
Generate printable analysis	AI Report	Select supplier, customer, or product and generate a printable report.
Manage staff access	User Access	Create users, assign roles, and review permissions.
Change interface language	Settings	Switch the user interface between English and Thai.